

University of Stuttgart
Institute for Signal Processing and System Theory
Professor Dr.-Ing. B. Yang



Masterarbeit D1486

LyMe: Lyrics-to-Melody translation using transformer architecture

Author: Swarnendu Sengupta

Date of work begin: 01 November, 2023

Date of submission: 01 May, 2024

Supervisor: Dr.-Ing. Stefan Uhlich, Giorgio Fabbro, Prof. Dr.-Ing. Bin Yang

Keywords: Transformer, Positional encoding, Gradio, CREPE, PESTO, Pitch, Music, BERT, Whisper, MIDI, RoPE

Abstract

Music is an integral part of our lives. The generation of music is a complicated task. A person needs to take into account the lyrics, tune, chords, and other features of music to generate it. However, with the growth of generative Artificial Intelligence (AI), there have been a lot of different approaches which have been allowed to generate music. But, the music generated by them is highly restrictive in many ways. Most of them are able to produce instrumental music like flute, violin or cello. Only a few are able to produce vocal music.

One key aspect of these systems are that they produce music from a text prompt description and not from lyrics. This thesis is aimed to solve this missing aspect by developing a system that takes the lyrics as an input and produces the vocal melody. This system that is developed by us is called LyMe which stands for Lyrics-to-Melody. For it to be developed, there is a need to find a dataset which has the vocals and corresponding text. However, there is no publicly available dataset. So, we create the dataset ourselves. In particular, we extract the vocals from songs and then pass it through various pitch detection and Voice Activity Detection (VAD) algorithms. The pitch is obtained in frequency (in Hz), which is converted to notes like C4.

From the songs, the lyrics are extracted. Then the objective becomes a translation of lyrics to musical notes. Since the transformer specialises in translation related tasks, it is selected as the base model. However, it is seen that the performance of a transformer could be improved by some modification to the architecture. The encoder of the transformer is replaced by a learnt language model which gives the embeddings of the words. However, the normal lyrics have missing some information of the song like Beats Per Minute (BPM). This information is extracted from the vocal song and fed into the model as additional information. Further, it is seen that with the quantity and quality of data available, the model is overfitting. In order to reduce this, some steps are taken like reduction of the parameters of the transformer, and increasing the amount of data. Additionally, the positional embedding layers of the original transformer is also changed to see the impact it has on the output of the model.

Towards the end of the thesis, it is seen that for this model to be used for analysis and generation, the position of the word in the song is also needed. For this, the output of the transformer is adapted to obtain two outputs: the notes and the position of the words that are to be sung. This helps the user to understand which word would be sung when and for how long. This also helps us to create a better Text-to-Singing Voice system. Some ablation studies are also conducted with regards to the type of modification done on the data. The thesis concludes by providing an application in Gradio where the user could provide some lyrics and create music by selecting some of the features.

Contents

Acronyms	V
Notations	VI
1. Introduction	1
1.1. Motivation and problem statement	1
1.2. Proposed solution	2
1.3. Thesis structure	3
2. Literature Review	4
2.1. Pitch detection algorithms	4
2.1.1. CREPE	4
2.1.2. PESTO	5
2.1.3. Basic Pitch	6
2.2. VAD algorithms	8
2.2.1. CREPE and PESTO	8
2.2.2. Basic Pitch	8
2.2.3. Silero	8
2.2.4. PyAnnote	9
2.3. Audio to Text conversion algorithm	10
2.3.1. Whisper	10
2.4. Transformer	12
2.5. Positional encoding	14
2.5.1. Inherent positional encoding	14
2.5.2. Linear positional encoding	15
2.5.3. Sinusoidal positional encoding	16
2.5.4. Rotational Positional Embeddings (RoPE) positional encoding	19
2.6. State-of-the-art (SOTA) models for transformations	20
2.6.1. Jukebox	21
2.6.2. WaveLM	21
2.6.3. MuLan	22
2.6.4. AudioLM	22
2.6.5. AudioGen	24
2.6.6. Riffusion	25
2.6.7. MusicGen	25
2.6.8. Moûsai	26

2.6.9. Mustango	26
2.6.10. M ² UGen	28
3. Proposed algorithm	31
3.1. Pitch detection for Singing Voice Melody Extraction	32
3.2. VAD algorithms - comparison	33
3.3. Audio-to-text with word timestamping	34
3.4. Pitch shifter and BPM	35
3.5. Positional encoding algorithms	35
3.5.1. Sinusoidal positional encoding	35
3.5.2. Rotary positional encoding	36
3.6. Data	36
3.6.1. Data processing	37
3.6.2. Models and data associated with them	40
4. Results	46
4.1. Models	46
4.1.1. Base model	47
4.1.2. Base model with padding information	49
4.1.3. Base model with padding information and additional loss function . .	51
4.1.4. Base model with word positioning	53
4.1.5. Base model with 0-1 word positioning	55
4.1.6. Base model with special character separation	56
4.1.7. Base model with special character separation and 0-1 word positioning	57
4.1.8. Base model with special character separation and 0-1 word position- ing with wet dataset	60
4.1.9. Compilation of all models	60
4.2. Application	61
4.3. Issues	63
5. Summary	65
Bibliography	66
A. Appendix	71
A.1. Mathematical explanation of RoPE	71
A.2. Pitch detection algorithmic comparison	73
A.3. Model Summary	73
A.4. Note frequencies	73
A.5. Base model - Graphs with sinusoidal positioning	79
A.6. Base model with padding - Graphs with sinusoidal positioning	79
A.7. Base model with padding and attention mask - Graphs with sinusoidal posi- tioning	79

List of Tables

2.1. Specifications of different Whisper models	10
3.1. Scale to MIDI Number Mapping	41
4.1. Model Parameters and Performance	62
A.1. Model Summary	74
A.2. Note Frequencies	75

Acronyms

AI	Artificial Intelligence
ASR	Automatic Speech Recognition
BPM	Beats Per Minute
CNN	Convolutional Neural Network
CQT	Constant-Q transform
DL	Deep Learning
LLM	Large Language Models
MIDI	Musical Instrument Digital Interface
ML	Machine Learning
RMS	Root-Mean Square
RNN	Recurrent Neural Network
RoPE	Rotational Positional Embeddings
RVC	Retrieval-based Voice Conversion
SOTA	State-of-the-art
SSL	Self-Supervised Learning
STFT	Short-time Fourier Transform
STT	Speech-to-Text
VAD	Voice Activity Detection

Notations

x	scalar variables, in time domain
$\mathbf{x}$	vector variables: boldface upright lower case letters
$\mathbf{X}$	matrix variables: boldface upright upper case letters
$x(t), H(z)$	functions of continuous variables: argument is placed in round parentheses
x_k	scalar elements of a vector or element of a sequence: element index is a subscript
$\mathbf{X}^T, \mathbf{x}^T$	transpose of the matrix or the vector
$\text{vec}(\mathbf{X})$	conversion of the argument $\mathbf{X}$ into a vector
X^H	Hermitian of $\mathbf{X}$
$a^*(\theta)$	complex conjugate of $a(\theta)$

1. Introduction

With the advent of technological advances in the field of generative AI, there has been a lot of instances where the quality of images, audio and video generated has been of a far superior quality to those that were produced even 2 years ago. The field of generative AI is very subjective and the results obtained from these complex architectures are subject to human scrutiny for their authenticity and likeability. In the previous years, the results produced were not so good and popular for usage. However, with evolution of various complex Machine Learning (ML) architectures and Deep Learning (DL) techniques, the results obtained now are not only better, but sometimes as good as humans could have created them. Sometimes, it becomes very difficult to differentiate between the "fake" media created as a result of these algorithms and the "real" media. This breakthrough has occurred in various media forms such as images, video and audio. Some of the most prominent examples would be of SORA [1], JUKEBOX [2] and many more.

One important form of media that is very crucial for the well-being of human life is songs and music. It helps in expression of feelings while conveying it in various forms based on factors like genre, tempo, beats, sound quality, words and other factors. Songs, in general, are composed of two major parts :- instrumentals, which is the background music composed of sounds produced from instruments, and vocals, which can be identified as singing voice with certain tonality, pitch, frequency and tempo.

The various generative models have led to believe that there is a possibility to generate any form of media through the use of AI and various ML algorithms. There also have been techniques which help in style transfer voices. One major example is Retrieval-based Voice Conversion (RVC) systems. This is a major breakthrough and helps changing the voice of one person to another, given the model is trained on the other person's voice. One example would be that the model is trained with a lot of data on the voice of a celebrity like Barack Obama. During inference, another voice from another person like Tom Cruise is provided to the model which would be converted such that it would seem as if Barack Obama said it.

1.1. Motivation and problem statement

Being a lyrics writer myself, it is easy for me to write lyrics in the form of poetry which has a certain rhyming scheme and beat. However, with very low knowledge of music it gets very difficult for me to be able to convert it to a song and hear it. Hence it is imperative to create a system which could understand and generate vocal music given the lyrics. This

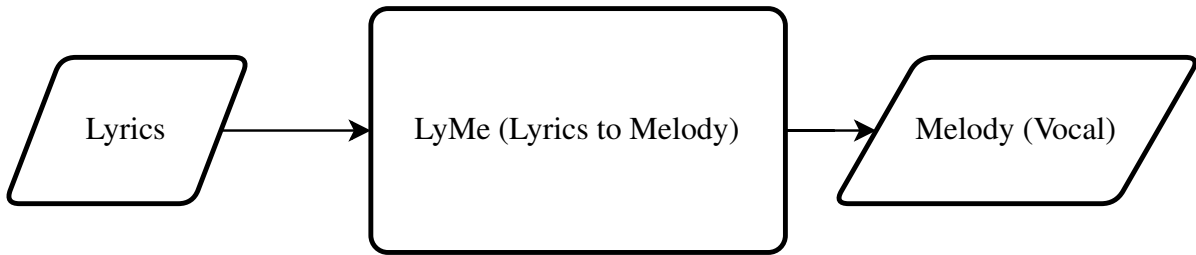


Figure 1.1.: The basic structure of the model which shows the input and the output(s) of the model. This does not show the internal structure of LyMe

this thesis attempts to lay a foundation to solve this problem. For the purpose of this thesis, we would focus on generating the vocal melody and not the vocals themselves, based on the words and other information given to the ML system as input. This translates to a sequence-to-sequence translation problem where the input sequence would contain the lyrics of the song and the output sequence would have the vocal notes. However, it is not just a seq2seq model which would just produce some vocal notes, but LyMe should be able to understand the difference between a love song, a rock song and a jazz song from the lyrics and other information provided to it.

The lyrics of songs could easily be obtained from generative Large Language Models (LLM) like ChatGPT [3]. The output of the model should produce a vocal melody which would be inline with the lyrics of the song. This could serve as a baseline for lyricists to understand how their lyrics should play under different conditions.

1.2. Proposed solution

An approach followed by the generative tools is to transform the audio data to mel spectrogram. The operations are then performed on the mel spectrogram. After the operations and mappings, they are converted back to the desired domain by using learnable reverse mapping functions. These functions are either already available like HiFi-GAN [4] or are learnt as a part of the ML pipeline. This is done because the transformations with mel spectrogram is well-established. This can be seen in the audio-to-audio domain, as well as in the text-to-video domain. However, there is very little research on keeping the data in textual media and then processing it further. Some of them include text-to-text, where the data is not converted into any spectrogram and is processed in textual format. This is the basis of many LLM foundation models like BERT [5], ChatGPT [3] and Llama 2 [6].

This thesis draws inspiration from this and converts vocal melody into textual representation of notes and processes it as a text-to-text-to-audio pipeline. Some pre-existing models and pre-processing steps have been used for the creation of the pipeline. Since there was no labelled data available for this problem, some autolabelling techniques had to be researched. The basic pipeline which shows the input and output of the thesis is shown in Figure 1.1.

1.3. Thesis structure

This thesis is divided into various chapters for easy understanding and readability. Chapter 2 covers some of the old techniques and SOTA that have been established and are closely related to our topic. It also contains some of the algorithms and details that have been used as a base or prior knowledge for the algorithms that were used in the thesis. The description of the algorithms that were used can be found in Chapter 2. This chapter consists of various topics which include the usage of various VAD and pitch detection algorithms, the description of transformers, and some SOTA architectures.

The comparison of these algorithms along with their areas of usage form a part of the results which can be found in Chapter 3. Chapter 4 contains the various results that were obtained during the whole project through different models by having different changes made to either the data or the model. It terminates by introducing an application that was created and also by highlighting some of the issues that were faced. Chapter 5 concludes the thesis, summarising the work done and also proposes other ways to approach this problem which might be a research topic for the future.

2. Literature Review

This chapter deals with the previously established theory needed for the implementation of the project. It encapsulates the basic building blocks on which other techniques and the algorithms were applied.

This chapter is further divided into various sections. Section 2.1 covers different algorithms that are present in literature to determine the pitch of an audio sample. VAD algorithms are covered in Section 2.2. Section 2.3 refers to the algorithm used to extract text from the audio. These three sections constitute the main data processing part of the thesis.

Section 2.4 covers the basic working principle of a transformer. Section 2.5 introduces us to various positional encoding techniques. The pros and cons along with its usability are also discussed. The chapter ends by giving an overview of the various SOTA models in Section 2.6 which have been used in various applications in generative AI in the field of audio and music generation.

2.1. Pitch detection algorithms

Pitch refers to the frequency of the audio at a given instance of time. There are various pitch determination algorithms. We would be discussing three of them which were compared. They are CREPE [7], PESTO [8] and Basic Pitch [9]. Here, their basic architecture, some key points, novelty in them and some other features have been discussed. When they were applied to different audio samples, the results are presented in Chapter 3.

2.1.1. CREPE

CREPE [7] is a deep Convolutional Neural Network (CNN) which operates on time-domain input. The objective of CREPE was the estimation of pitch or the fundamental frequency (f_0) of a monophonic audio signal. It was the SOTA technique in 2018 when it outperformed many heuristic approaches like pYIN and SWIPE [10, 11]. It was trained on RWC Music database [12] and MedleyDB [13], using a 1024 raw sample input of 16 kHz sampling rate followed by 6 1D convolutional layers to output a 360-dimensional output vector. This output corresponded to a specific pitch value which is mapped to the scale between C1 and B7, corresponding to 32.70 Hz and 1975.5 Hz using the Equations 2.1 and 2.2 .

$$\mathbf{c}(f) = 1200 \cdot \log_2 \frac{f}{f_{ref}} \quad (2.1)$$

$$\hat{c} = \frac{\sum_{i=1}^{360} \hat{y}_i \mathbf{c}_i}{\sum_{i=1}^{360} \hat{y}_i}, \quad \hat{f} = f_{ref} \cdot 2^{\hat{c}/1200} \quad (2.2)$$

where $\mathbf{c}_i$ refers to the cent value of the i^{th} node and f_{ref} is the reference pitch which was set to 10 Hz.

After this, Gaussian blurring was applied so that the near-misses in the truth values are softly penalized and the huge misses are penalized exponentially. At the end, the network was trained to minimize the binary entropy loss for each vector.

CREPE was trained mainly on instrumental data and evaluated on raw pitch accuracy and raw chroma accuracy. To test for the robustness of the system, they downgraded the samples using Audio Degradation Toolkit [14] by introducing various types of noise. It showed that it was able to predict the pitch or the f0 frequency of a given sample more accurately than pYIN and SWIPE.

At the end, CREPE provides three outputs - the first contains the starting timestamps, the second contains the predicted fundamental frequency in Hz, and the third contains the confidence, i.e. the confidence in predicting the value of the pitch.

2.1.2. PESTO

PESTO [8] approaches the problem of pitch estimation using Self-Supervised Learning (SSL). They use a Siamese neural network [15] that takes pitch-shifted versions of the same audio that was represented in its Constant-Q transform (CQT). With this kind of architecture, there is a very high probability that the model would map all inputs to the same embedding. To prevent this from happening, they had introduced class-based equivariance loss. In order to construct the network lightweight and transposition-equivariant, Toeplitz fully-connected layers were introduced.

PESTO was published in 2023 and it compared its performance with CREPE [7], whose description can be found in Subsection 2.1.1. They also compared it to SPICE [16] which was also a self-supervised network. It also took the input from the CQT frames of the pitches and learnt the transpositions between the inputs. However, they achieved decent results as they have a decoder which takes the predicted pitch as an input and tries to reconstruct the original CQT frames. This could be seen in the Figure 2.1.

On the comparative study that was done among PESTO [8], CREPE [7] and SPICE [16], it was seen that CREPE performed better. The main reason for it to perform better was that CREPE had a larger model and it was trained in a supervised manner on a lot of data. Hence, it performed better than either SPICE or PESTO. PESTO was compared with SPICE and DDSP-inv [17] which were also based on SSL. PESTO was found to provide better results.

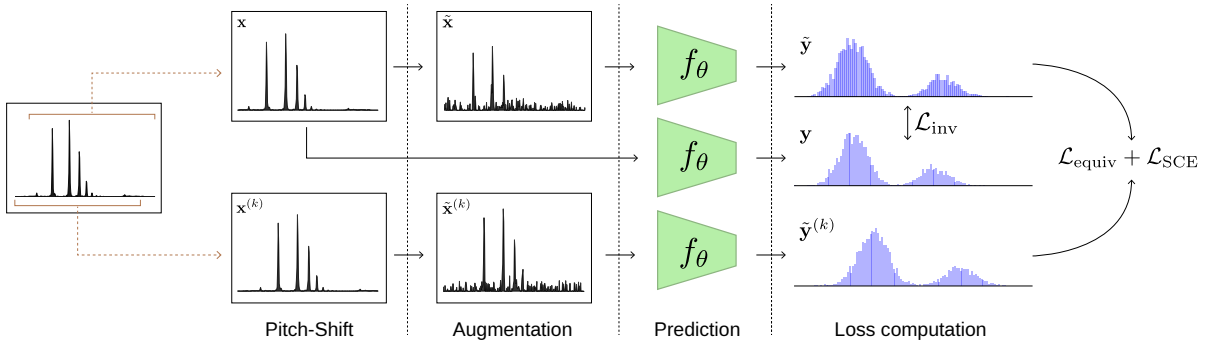


Figure 2.1.: Working of the PESTO from [8]

The network was trained on vocals as well as monophonic instrumental dataset, which made it one of the ideal choices for this thesis. PESTO, like CREPE, provides three outputs, namely timestamps, pitch frequency in Hz and confidence of predicting the frequency.

2.1.3. Basic Pitch

Basic Pitch [9] was developed by Spotify. It was proposed as a lightweight neural network for musical instrument transcription, which supports polyphonic outputs and generalizes to a wide variety of instruments. It was also trained on vocals. The model was trained to predict frame-wise onsets, multipitch and note activation. This made it possible to be used in the context of polyphonic sounds.

The main aim of Basic Pitch was to make it memory efficient as well as accurate in terms of its predictions. It used several techniques like harmonic staking, harmonic CQT transforms, convolutions, along with producing three different output posteriorgrams which provided the information of the onset of a note, the duration of the note, and the duration in which the pitch is active. The architecture of this can be found in Figure 2.2. It was assumed that the output posteriorgram which predicted the pitch would be the one which would be the closest to the actual audio. The usage of convolution was to have a denoising effect on the data.

The training was also done in a novel way. Binary cross entropy was used as a loss function for each of the output posteriorgram. The input was restricted to 2 seconds and sample rate of 22050 Hz. To ensure that the data came from various sources, had variety in themselves, had also a variety of instruments and other factors like quality of recording and publishing, the data was used from various sources.

Basic Pitch provides three types of outputs, namely the raw model inference of the audio, Musical Instrument Digital Interface (MIDI) file and note events based on the model output. The way it has been used can be found in more details in Chapter 3.

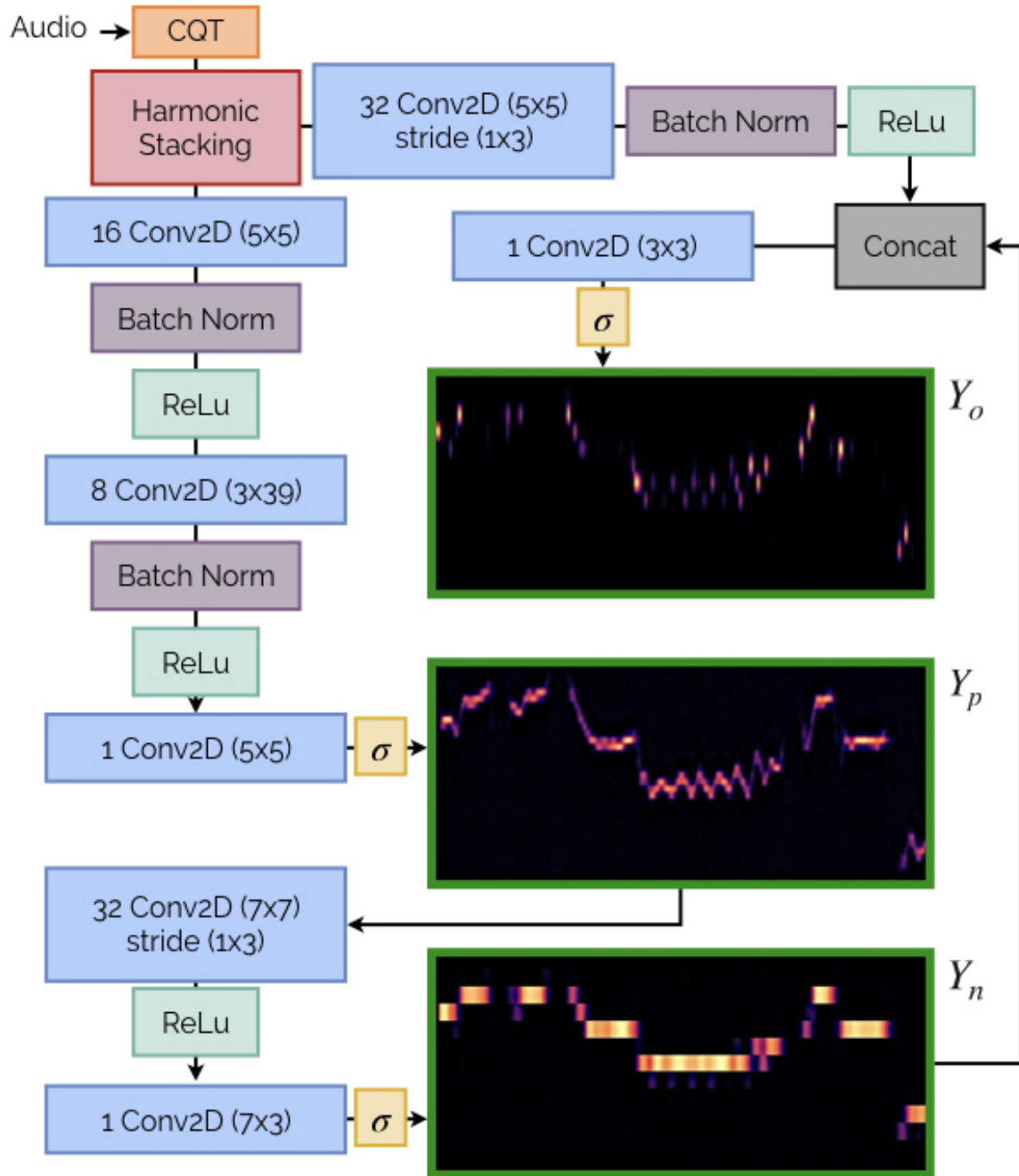


Figure 2.2.: Architecture of Basic Pitch taken from [9]

2.2. VAD algorithms

Since we are dealing with vocal melody, it is of extreme importance to know when the vocals are present in the audio and when it is absent. Since the pitch detection algorithms can only detect pitches of audio, there could be instances where noises or other artifacts could have been predicted. Hence, we need voice activity detectors to identify the portions of the vocal music where the vocals were active. Here, some of the major VAD algorithms have been discussed along with their methodology and outcomes. The comparative study of them have been done in Chapter 3.

2.2.1. CREPE and PESTO

The algorithms and structure behind CREPE and PESTO have been discussed in Subsections 2.1.1 and 2.1.2. CREPE and PESTO provide us with the predicted frequency of the audio, the timestamp of the occurrence of the frequency and also the activation or confidence of the estimated pitch or frequency at that timestamp. The confidence output of these models were used to estimate the VAD. The details on how this was used could be found in Chapter 3.

2.2.2. Basic Pitch

Basic Pitch algorithm is a bit complex when it comes to analysis and extraction of VAD. The algorithm provides three types of outputs: one is the raw output of the model, second is a MIDI file, third is a list of note events based on the model output. Due to its capacity to predict polyphonic sounds, it is very important to make sure that during one time instance only one note is being played.

For the purpose of VAD, the time instances in the MIDI files where no sound was being produced, were determined to be silences. In the MIDI file, there can be instances where polyphonic sounds are present. Each instrument's active regions are found out. The timestamps of silence are the timestamps on which none of the instruments are active. We achieved this by looping over all the instruments and marking the timestamps in which the instruments were being played as 1 and 0 for the timestamps where no instruments were active. This indicated the areas of silence and gave us the desired VAD output from the algorithm.

2.2.3. Silero

Silero [18] was meant for speaker segmentation. It is a model which performs VAD, speaker change detection and overlapped speech detection simultaneously. The model was trained as a multi-label classification task using permutation-invariant training [19]. It works on audio chunks of 5 sec with a resolution of 16 ms. It incorporated data augmentation using additive background noise with random signal-to-noise ratio. Another augmentation technique that

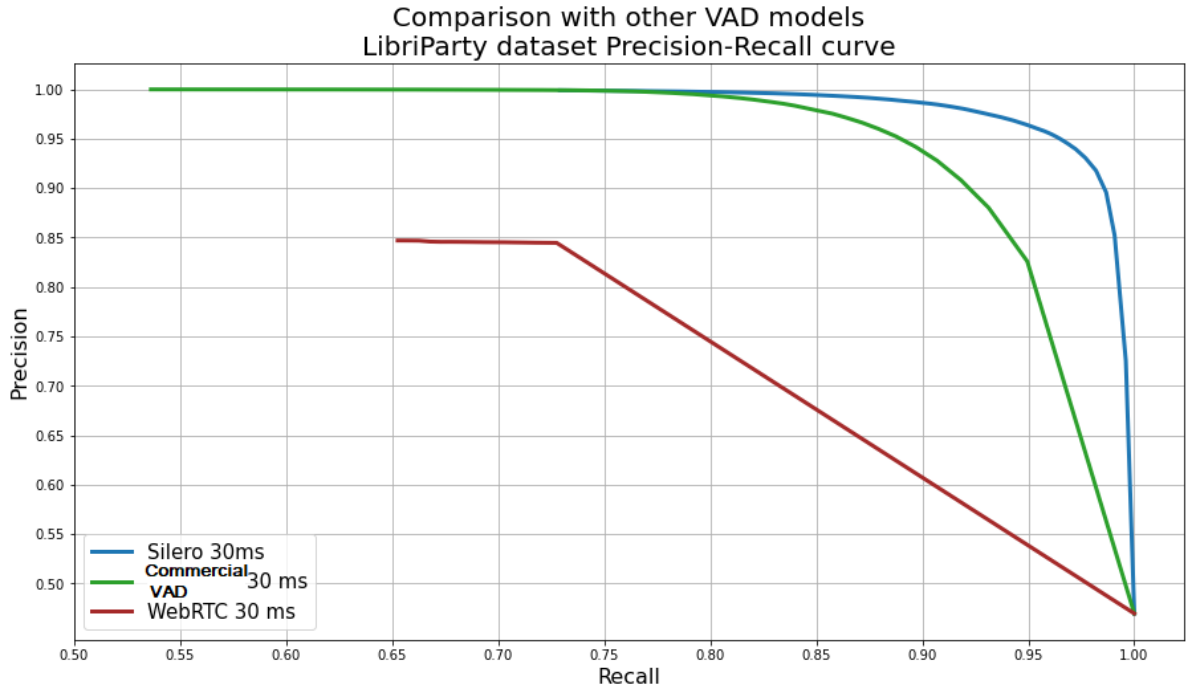


Figure 2.3.: Comparison of Silero with others

was used was addition of two random audio samples with random signal-to-signal ratio. The segmentations were also merged accordingly.

It uses a multi-head attention based neural network which gives the output of a segment as a probability of speech or not. This is used against a pre-defined threshold for the determination of speech or noise. It has been noted that it has better precision and recall scores when compared to other VAD models like SpeechBrain, WebRTC and a commercial model 2.3. The model also performs well on audio files from different domains with various background noises and quality levels.

2.2.4. PyAnnote

PyAnnote [20] VAD outputs $y(t) = 0$ if there is no speech at time t and $y(t) = 1$ if there is speech at time t . Timestamps with prediction scores greater than a tunable threshold Θ_{vad} are marked as speech.

PyAnnote for voice activity detection is compared to models that are specifically trained for voice activity detection and it shows that it achieves better results on false alarm rate, missed detection rate, precision, recall, and F1-score [21]. It uses a Recurrent Neural Network (RNN) which is a simpler version of the implementation of the RNN mentioned in [22]. It has shown to have better precision, recall and F1 score as shown in Table 1 in [20].

2.3. Audio to Text conversion algorithm

One of the major aspects of this project is to associate text to music. We need an algorithm which would have the ability to convert vocal music to text. Since vocal music can be closely associated to speech, we would need an Speech-to-Text (STT) algorithm. There are several algorithms like wav2vec2 [23] which are capable to producing text from speech. However, for the purpose of this project Whisper [24] is used as it is one of the SOTA algorithm for performing STT.

2.3.1. Whisper

Whisper [24] is a SOTA and innovative Automatic Speech Recognition (ASR) model developed by OpenAI. It consists of an encoder-decoder transformer architecture [25]. One of the major reasons for the model's efficacy is its extensive training on an impressive dataset of 680,000 hours of multilingual audio of speech [26, 27]. This dataset includes a vast 540,000 hours of English speech and a substantial 117,000 hours spanned across 96 diverse languages [28]. This highlights one of the major building blocks of supervised machine learning, which is the availability of a well-labelled dataset.

Along with ASR, it can perform various other tasks like language identification, VAD, transcription and translation. OpenAI released various models based on the different parameters which can be seen in Table 2.1.

The working of Whisper can be shown in Figure 2.4. There are some short-comings that

Model	Layers	Width	Heads	Parameters
Tiny	4	384	6	39M
Base	6	512	8	74M
Small	12	768	12	244M
Medium	24	1024	16	769M
Large	32	1280	20	1550M

Table 2.1.: Specifications of different Whisper models

were identified with Whisper for music data. It has been trained on speech data which is significantly different than music dataset. The execution time of Whisper is almost double than real-time. In order to transcribe 20 seconds of audio, it requires close to 35 seconds on an average. There are also instances where Whisper, during ASR of speech data, hallucinates and produces words not present in the speech. This is seen in manifold in music data as music is a special type of speech with a lot of complications and features that needs to be addressed differently.

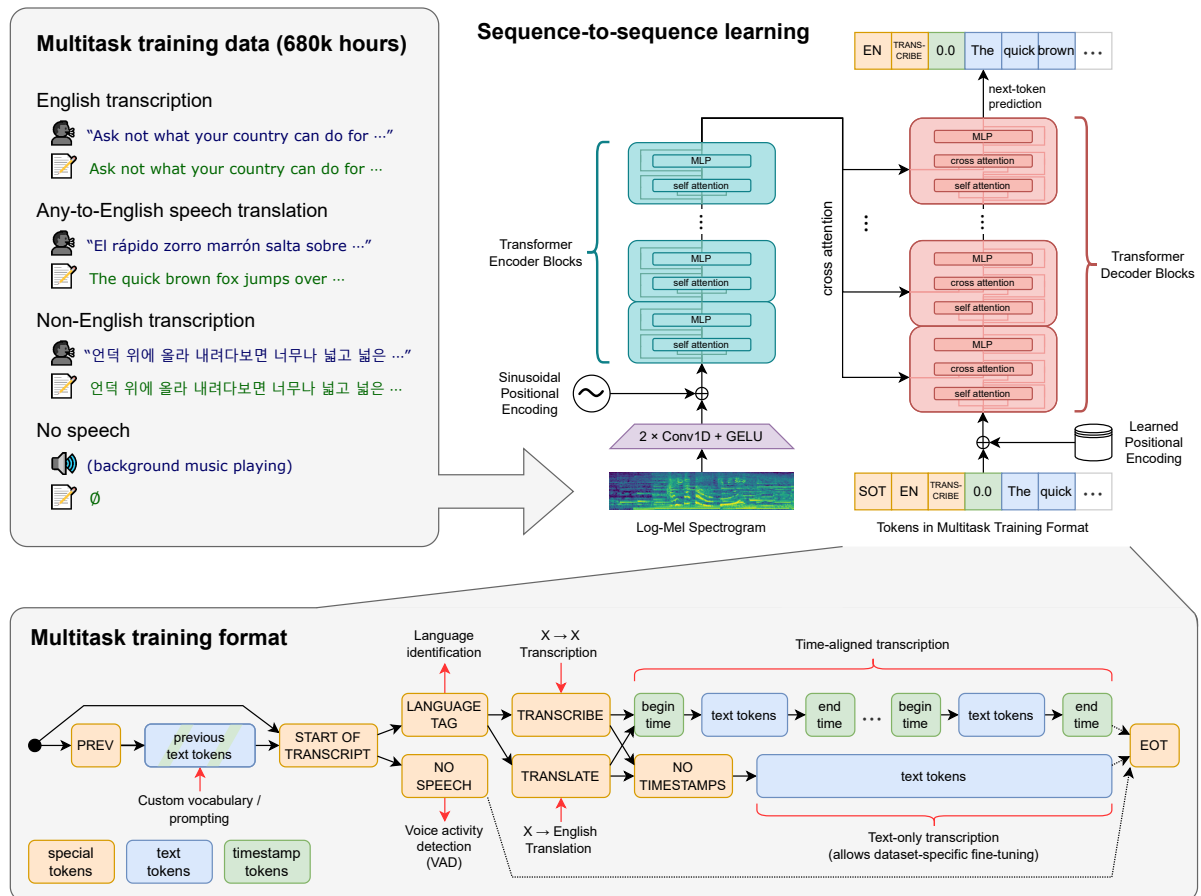


Figure 2.4.: Overview on the approach to train the Whisper model

2.4. Transformer

Transformer [25] is a deep learning model that specialises in machine translations, language modelling and other seq-to-seq tasks. The basic structure of a transformer consists of an encoder-decoder structure as can be seen in Figure 2.5. It consists of an encoder (which is the right side) and a decoder (which is the left side). The encoder consists of

- **Input embedding** : The input to the model is usually a set of tokens. However to understand how semantically close they are to each other, there is a need to do an embedding.
- **Positional information** : Since the transformer does not have any inherent notion of sequential order, positional encodings are added to the input embeddings to provide information about the position of tokens in the sequence. There are various types of positional encodings that can be used and are described in Section 2.5.
- **Encoder stack** : This consists of a multi-head self-attention mechanism. Here, each token attends to every other token in the input sequence. This allows the model to weigh the importance of different tokens. This is done in parallel with different learned linear projections which allows the model to attend to different aspects of the input sequence simultaneously.
- **Feedforward Neural Network**: After the attention mechanism, the output is passed through a feedforward neural network.
- **Layer Normalization and Residual Connections**: Both the multi-head self-attention mechanism and the feedforward neural network are passed through layer normalization and residual connections. Layer normalization helps stabilize the training process by normalizing the activations of each layer. Residual connections facilitate the flow of gradients during training, which helps mitigate the vanishing gradient problem.

The decoder layer is very similar to the encoder layer with a few modifications:

- **Multi-Head Self-Attention Mechanism**: Similar to the encoder stack, the decoder stack also performs multi-head self-attention. During decoding, attention is masked to prevent tokens from attending to subsequent tokens in the sequence, ensuring that the model generates each token autoregressively. In most cases the masking is done to disable the model from looking into the future and allowing it to only see the past.
- **Multi-Head Cross-Attention Mechanism**: In the decoder, there is also a multi-head cross-attention layer which attends to the encoder's output. This mechanism allows the decoder to focus on relevant parts of the input sequence when generating each token.
- **Feedforward Neural Network**: The output of the cross-attention mechanism is passed through a feedforward neural network, similar to the encoder stack.
- **Layer Normalization and Residual Connections**: Just like in the encoder stack, layer normalization and residual connections are applied after each sub-layer in the decoder stack.

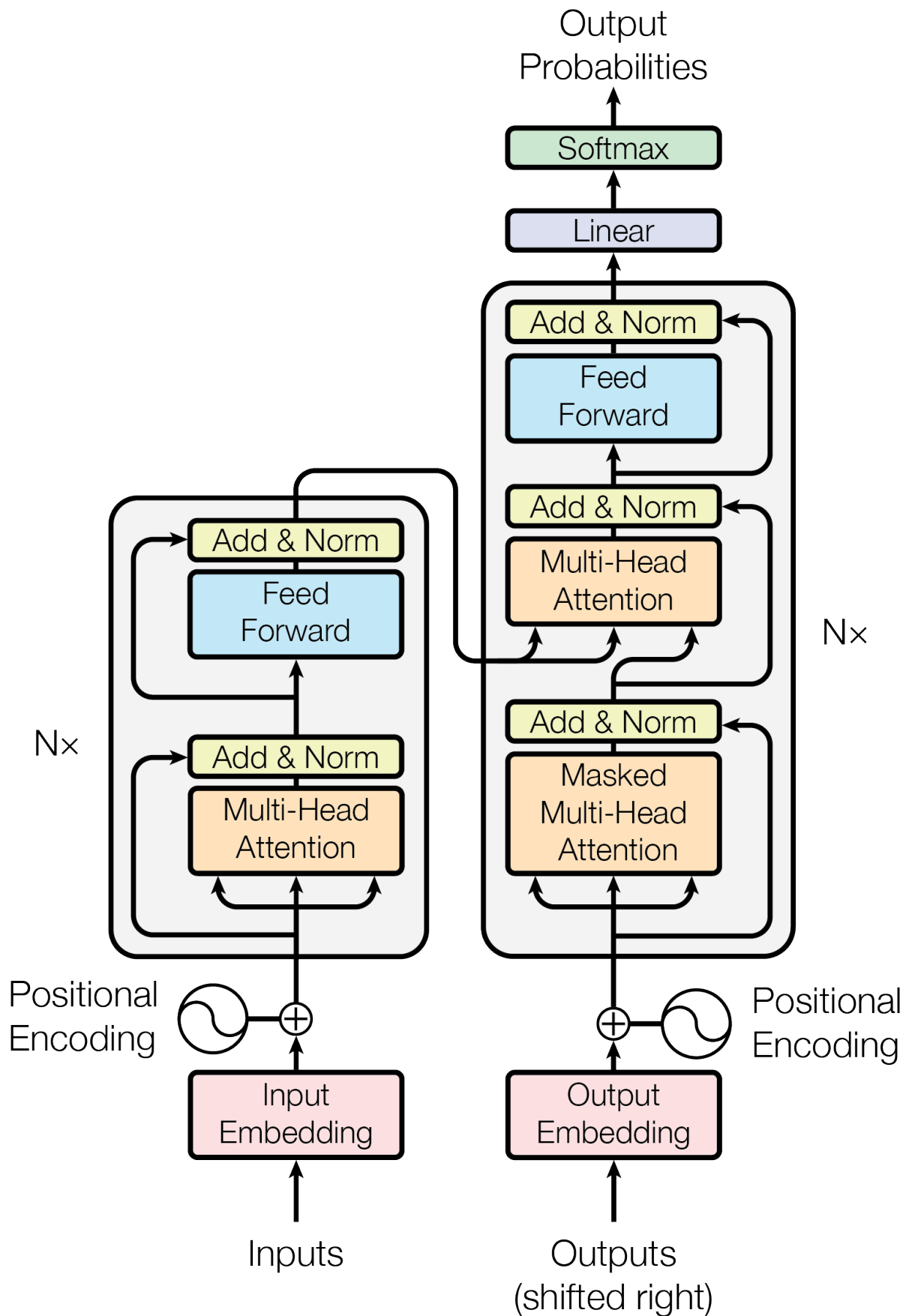


Figure 2.5.: Basic transformer architecture

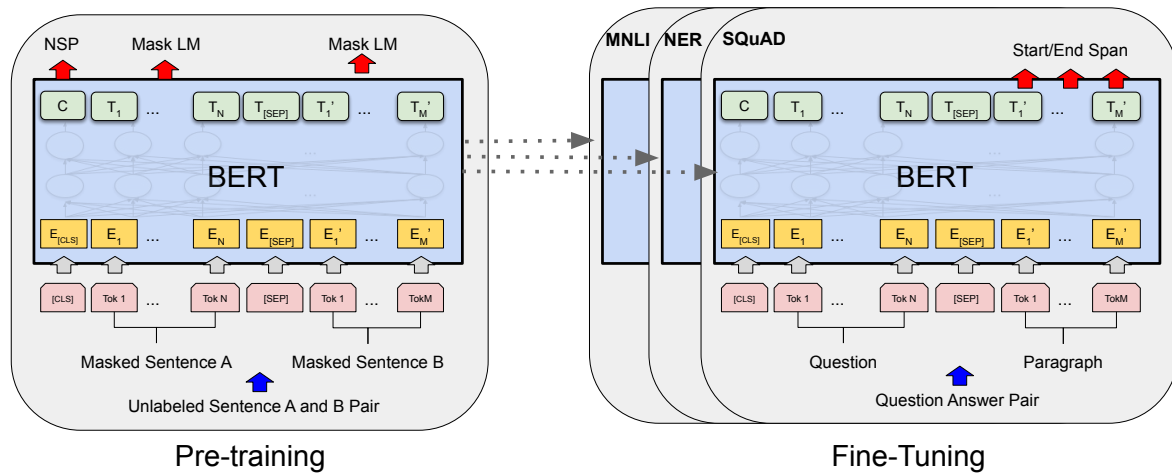


Figure 2.6.: BERT pre-training and fine-tuning procedures

The output of the decoder stack is passed through a linear transformation followed by a softmax function to obtain the probability distribution over the vocabulary for each output token. This describes the basic architectural design of a basic transformer which also is the backbone of the project.

2.5. Positional encoding

When dealing with sequential data, there is a notion of applying positional encoding to the input such that the model can understand that it is a sequential dataset and not some random numbers without any meaning.

There are various types of positional encoding algorithms that are possible. Some of them have been described below.

2.5.1. Inherent positional encoding

Some of the models have inherent positional encoding which it has learnt during the course of its training. One such example is that of BERT [5]. BERT has been trained on a huge corpus of text. Initially they were randomly initialized.

However, BERT is pretrained on two unsupervised learning objectives: Masked Language Model (MLM) and Next Sentence Prediction (NSP). In MLM, a sentence with some tokens are given to BERT and is trained to find the missing tokens based on the surrounding texts or tokens. Here, BERT needs to understand the sequential ordering of the tokens or words so that the sentence makes sense. This can be seen in Figure 2.6 and Figure 2.7. In NSP, BERT is given a pair of sentences during training and is trained such that given the first sentence, it can predict the second sentence. This makes BERT understand the importance of positional

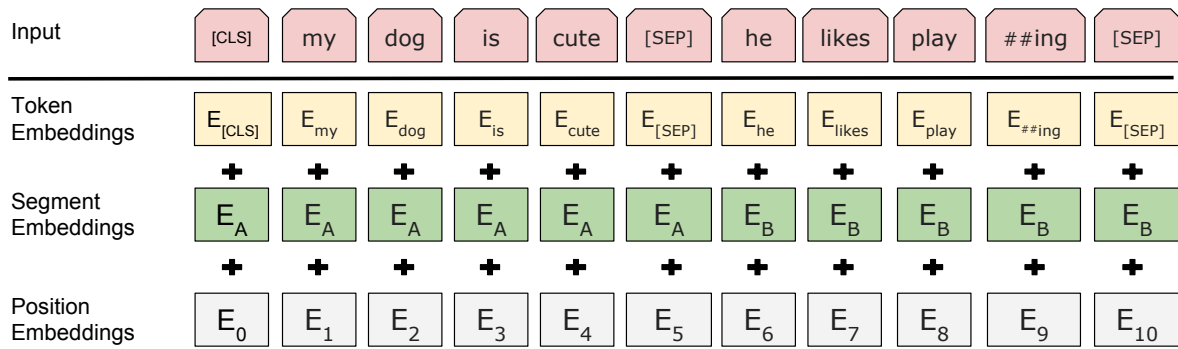


Figure 2.7.: BERT tokenisation and input representation. It shows that the input is a sum of three elements.

encoding of sentences as can be seen from Figure 2.7.

Thus, the type of positional encoding that is learnt and is not explicitly stated is termed as inherent positional encoding or learnt positional encoding. Learnt positional encoding is termed as positional embeddings. The BERT model that was used in this project, produces a feature vector of 768 for each token identified by it.

2.5.2. Linear positional encoding

Given that there is a need to do positional encoding, the simplest solution would be to introduce linear positioning where the first timestamp is assigned the number **1**, the second timestamp the number **2** and so on. There are numerous drawbacks to this approach:

- The value would become large for very large sequences. This might be even larger than the values that it would represent in the original series, and the model would focus more on this rather than the actual series.
- The model could possibly see larger sequences in the validation or inference than those during training. The model would fail in that scenario.
- The model might not have seen any training data for a specific length. This would hurt the generalization of the model and cause the model to either fail or give random outputs.

In order to prevent the issues listed in List 2.5.2, some criteria could be setup. The criteria could be :

- The output of the encoding should be unique for each timestamp. This means that the 1st position for every sample would be the same but would be different than any other position in any sequence.
- The distance between any two timestamps should be consistent across sequences with different lengths. An example would be that the distance between the 3rd timestamp and

the 5th timestamp would be the same for a sequence of length 10 or 100. This would indicate that the positional encoding is non-biased and unaffected by the length of the input sequence.

- The encoding should allow for larger sequence that it has not seen during training. However, the value should be bounded and not rise to infinity with the increase in the number of tokens in the sentence. This would help in generalization of the encoding scheme.
- The encoding scheme should be deterministic in nature. This ensures that the model is real and can be applied to encode positions and be fed to the model.

Linear positional encoding fails to satisfy most of these criteria.

2.5.3. Sinusoidal positional encoding

One of the earliest positional encoding mechanisms that was proposed in [25] is sinusoidal positioning. This is, however, not a single number. It is a d — dimensional vector that encodes the information about a specific location in a sentence. Another important aspect is that it is not integrated in the model. It is a separate mechanism which simulates the position of the token in a sequence. This helps in enhancing the model's input and helps it to understand the sequential nature of the input and the associated tokens.

The mathematical formulation for the positional encoding is given as :

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \\ \text{PE}(\text{pos}, 2i+1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \end{aligned}$$

In the original paper, d_{model} was 512. It can be derived from the function that the frequencies are decreasing along the vector dimension. Thus it forms a geometric progression from 2π to $10000 * 2\pi$ on the wavelengths. The information received from the positional encoder is added to the input word embeddings. To be able to do this, the dimensions of the positional encoding and the word embedding has to be same.

Figure 2.8 is a representation of a positional encoding. Since sinusoidal positional encoding method ensures that the nearby positions in a sequence have similar encoding values or not too distant values, there is a smooth transition of colors in the x-axis. This type of encoding also satisfies all the criteria given in List 2.5.2.

Though it seems to be the best as it solves the problem of positional encoding. However, there are a few drawbacks that it possesses. They have been listed below:

1. Fixed frequency component - Sinusoidal encoding provides fixed frequency components to the different dimensions of the encoding. Thus the encoding might not adapt to different types of sequence or might not be able to capture complex positional relationships which are beyond a certain frequency range.

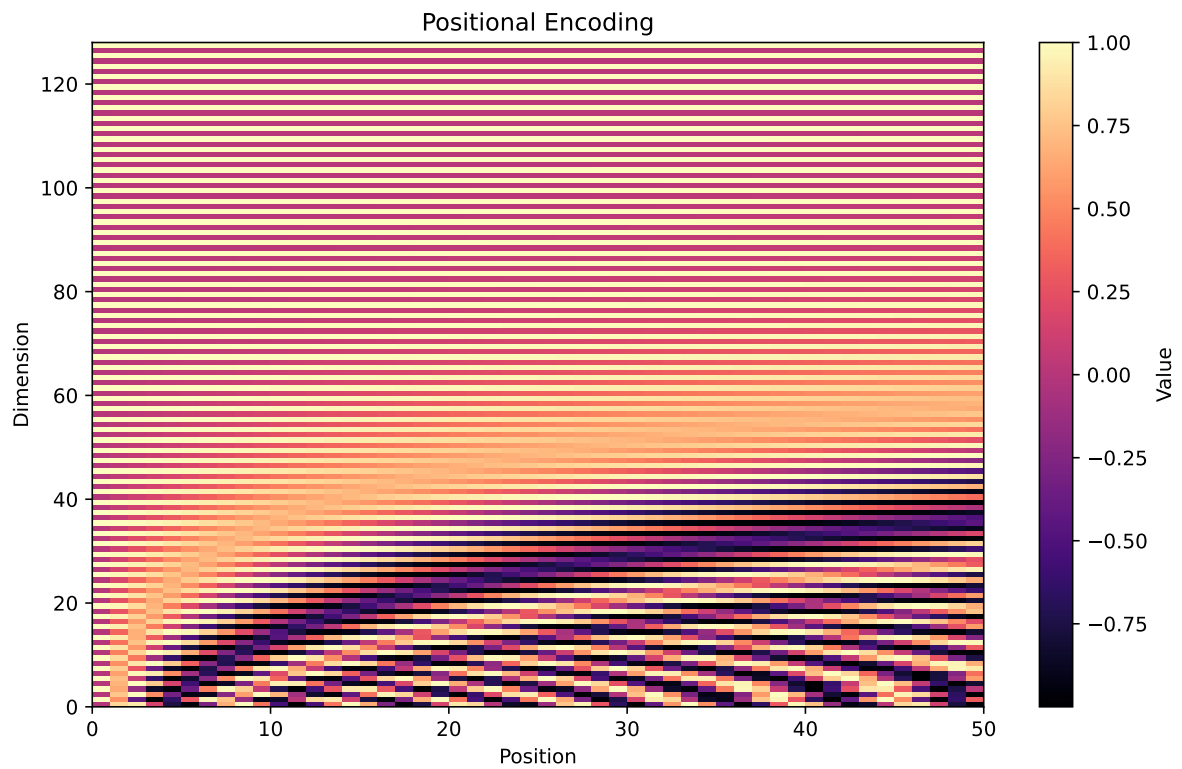


Figure 2.8.: The 128-dimensional positional encoding for a sentence with the maximum length of 50.

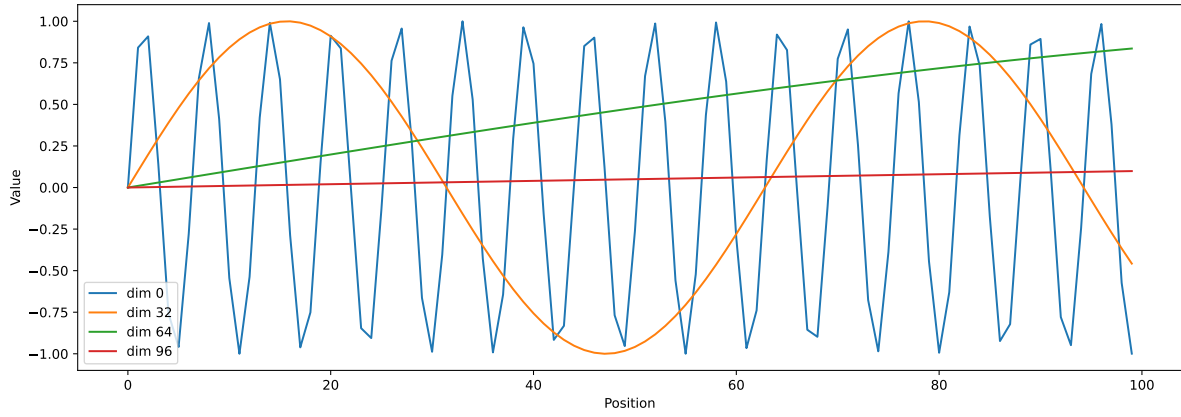


Figure 2.9.: Positional encoding with sinusoidal curves

2. Limited representation power - It can capture linear or periodic patterns well due to its sinusoidal components. However, it might fail to represent complex periodic patterns or dependencies that might be present in the sequence.
3. Difficulty with long sequences - When dealing with very long sequences, sinusoidal encoding might not be able to provide good representation of the positional information when the relative distance between the tokens in the real-world is very high, like from the start of sentence to end of sentence. The reason for this is the amplitude of the sinusoidal functions would decrease with increase in frequency which would result in weaker encoding of the signals for distant positions. This can be seen in the 1D case in the Figure 2.9. It can be seen that dim 96 has very high frequency and hence very low amplitude. At very long distance, this value would be almost negligible in comparison to other values of the other embedding of the sequence.
4. Interactability - Since the encoding treats each dimension separately and doesn't work on the interaction between them, it limits the ability to represent complex relations that might occur across dimensions.

The algorithm for getting the encoding is provided in Algorithm 1.

Algorithm 1 Sinusoidal Positional Encoding**Data:** Sequence of input tokens $x_1, x_2, \dots, x_n$ **Result:** Sequence of output tokens $y_1, y_2, \dots, y_n$ **Input:** Input embeddings $X = [x_1, x_2, \dots, x_n]$ **Output:** Output embeddings $Y = [y_1, y_2, \dots, y_n]$

```

 $Y \leftarrow X$  for  $i \leftarrow 1$  to  $n$  do
  for  $j \leftarrow 1$  to  $d_{model}$  do
    if  $j \bmod 2 = 1$  then
       $Y[i, j] \leftarrow \sin(\frac{i}{10000^{2j/d_{model}}})$ 
    else
       $Y[i, j] \leftarrow \cos(\frac{i}{10000^{2j/d_{model}}})$ 
    end
  end
end

```

Some of these challenges would be addressed in Section 2.5.4.

2.5.4. RoPE positional encoding

As introduced in the last part of Section 2.5.3, there are some problems with sinusoidal positional encoding. This led to another type of positional encoding which addresses some of these issues. Some of the features that RoPE possesses are:

1. Encoding function - Though RoPE also includes sine and cosine functions, there is a different formulation for the calculation of the encoding values. Each dimension represents a rotary function with a specific frequency, which varies across dimensions. This allows for a broader range of frequency components compared to sinusoidal positional encoding.
2. Rotary functions - The rotary functions possess different frequencies and hence capture wider range of positional relationships.
3. Adaptability - As the frequency components can vary across dimensions, the encoding scheme can be adjusted to better suit sequences with diverse or changing positional patterns. This makes it better to adapt to changing lengths and also adjust to lengths that the model has not seen.
4. Interactions - Each dimension has its own frequency, which allows for it to encode the interactions between the dimensions into positional representation

Given these benefits of RoPE positional encoding, it was also a potential candidate for this project. However, like each system, some limitations are also present for this type of positional algorithm:

1. Computational cost - RoPE incurs higher computational cost due to its calculative property of the angles.
2. Interpretability - The impact of the rotary functions on the encoding scheme is not intuitive in nature but it has a sound mathematical base which backs up the outcome.
3. Limitations to lengths - In the paper [29], the authors have not found a reason for the faster convergence of other models which incorporate other positional encoding schemes. Further, they found that even though RoPE has favourable property for long-term decay, they do not have any explanation for the fact that the model with RoPE performs better on longer texts than its peers.

The mathematical explanation of RoPE can be found in Appendix A.1. The algorithm for getting the encoding is provided in Algorithm 2.

Algorithm 2 Rotational Positional Encoding

Input: Input tensor $X = [x_1, x_2, \dots, x_n]$

Input: Sequence length seq_len

Output: Output tensor $Y = [y_1, y_2, \dots, y_n]$

Data: Position $(0, max_len)$

Data: Division term $exp(0, d_model, 2) * (-log(10000.0)/d_model))$

▷ This is the division term that would be used

$sin_theta = sin(Position * Division\ term)$

$cos_theta = cos(Position * Division\ term)$

$even_indices \leftarrow (0, X.size(-1), 2,)$

▷ Take all the even indices of the feature vector

$odd_indices \leftarrow (1, X.size(-1), 2,)$

▷ Take all the odd indices of the feature vector

$X_even \leftarrow X[even_indices] * cos_theta[: seq_length] - X[odd_indices] * sin_theta[: seq_length]$

$X_odd \leftarrow X[even_indices] * sin_theta[: seq_length] + X[odd_indices] * cos_theta[: seq_length]$

▷ Y is the same size as X and is the rotated matrix with positional embedding present in it

$Y[even_indices] \leftarrow X_even$

$Y[odd_indices] \leftarrow X_odd$

2.6. SOTA models for transformations

A lot of SOTA models are present that work on similar areas as the project has intended. Most of them work on the areas of music generation from text. However, the text that they use is generally a free flow text and not the lyrics of a music, and the music that is generated is instrumental and not lyrical music. However, they provide a way to look into the world of

music and help in conversion of text to music. Some of these SOTA models are explained below with some of their features and a brief of their workings.

2.6.1. Jukebox

Jukebox [2] provides one of the earliest attempts to generate music with singing from text in the raw audio domain. The approach was to train three separate VQ-VAE [30] with different temporal resolution. For each level, the audio was encoded into their respective latent vectors. These were then quantized to the nearest codebook vector. The code is a discrete representation of the audio on which the priors are later trained on. The decoder takes the sequence from the codebook vectors and reconstructs the audio. The main reason for separate VQ-VAE was to ensure different features were being emphasised on at different instances. It also provides various levels of conditioning to make it more controllable. The first level of conditioning was done by providing artist and genre labels for the songs. This would reduce the entropy of audio predictions which enables it to achieve better quality for a given style. Additionally, during generation time the model could understand the genre and provide songs pertaining to that genre.

Additional conditioning was done by providing a timing signal for each sample which includes information about the total duration of the song, start time of the sample and how much time has passed since the song had started. Further conditioning was done on lyrics. This task was achieved using Spleeter [31] and running NUS AutoLyricsAlign [32]. These helped in providing lyrics for a given chunk of audio accurately. After this, an encoder-decoder style model with a Transformer as an encoder and the decoder was a music decoder was used. The queries from the music token was only allowed to attend to the keys and values of the lyrics tokens.

The dataset used consisted of around 1.2 million songs which was paired with lyrics and meta-data from LyricWiki, which provided all the necessary details like artist, album, genre, year of release etc. The audio samples were 32 bit and 44.1 kHz in nature. Despite achieving such a system which provided human-like audio, the authors felt that there were noises which occurred due to the upsampling and downsampling processes. Also, given the autoregressive nature of the sampling process, a minute of rendition used to take about 9 hours. However, as the computing speeds and distillation of the model was done, the model provides faster results but the audio is still grainy and contains noise.

2.6.2. WaveLM

WaveLM [33] was a step towards SSL which focused on learning universal representation for all speech tasks. It was trained to jointly learn masked speech prediction and denoising in the pre-training stage. Additionally, it provides gated relative position bias for the Transformer structure, which enables it to emphasise the sequential ordering of the input speech.

The model structure involves a Transformer backbone. It consists of a convolutional feature encoder whose output is then fed to a Transformer encoder. The convolutional encoder has

temporal convolutional layer followed by layer normalization and a GELU activation layer. For the improvement of the model, a gated relative position bias was used which was based on the offset between the "key" and "query" in the self-attention mechanism of the Transformer. Additionally, the masked speech denoising and prediction framework was proposed which helped to increase the model robustness of complex acoustic environments but also preserved the speaker identity.

The WavLM Base and WavLM Base+ have 12 Transformer encoder layers, 768-dimensional hidden states, and 8 attention heads, resulting in 94.70M parameters. The WavLM Large has 24 Transformer encoder layers, 1024-dimensional hidden states, and 12 attention heads, resulting in 316.62M parameters. The relative position embeddings are shared across all layers, which avoids significantly increasing the number of parameters. It was primarily pre-trained on LibriSpeech dataset. The downstream tasks were based on SUPERB, which provides a testbed for various speech related tasks. They include Speaker Identification, Automatic Speaker Verification, Speaker Diarization, Phoneme Recognition, Automatic Speech Recognition, Out-Of-Domain Automatic Speech Recognition, Keyword Spotting, Query by Example Spoken Term Detection, Speech Translation, Intent Classification, Slot Filling, Emotion Recognition, Speech Enhancement, Speech Separation and Voice Conversion. It was shown that WavLM large performs better than HuBERT Large [34] on all the tasks.

2.6.3. MuLan

MuLan [35] is a first attempt which links music audio to natural music descriptions. It takes the form of a two-tower, joint audio-text embedding. The goal was to create a shared embedding space for music and text such that the proximity is predictive of the shared semantics within and across modalities. It used a cross-modal contrastive learning approach on a simple two-tower architecture. The two towers or networks share no weights, but terminate to a normalized embedding space with the same dimensionality. The audio embedding network takes in log mel spectrogram context windows as input whereas the text embedding network takes null-padded text as input. It can be seen visually in Figure 2.10.

For training purpose, a batch-wise Contrastive Multiview Coding loss function is used. Audio Spectrogram Transformer (AST) and Resnet-50 were used for the purpose of audio embeddings. The parameters of AST include a stack of 12 Transformer blocks that are applied to the log mel spectrogram context windows. For the word embeddings, BERT-base-uncased was used. The training data was scraped to obtain nearly 370K hours of music.

2.6.4. AudioLM

AudioLM [36] proposed a framework to produce realistic audio or piano music by listening to a small sample of either of them. The audio generated demonstrates long-term consistency and high fidelity. It identified the problem that text to audio and normal audio processing possesses. It identified that text could be identified by a few dozen characters, audio consists of a few hundreds of thousands of values in a very small span. Also, text to audio is a one-to-many

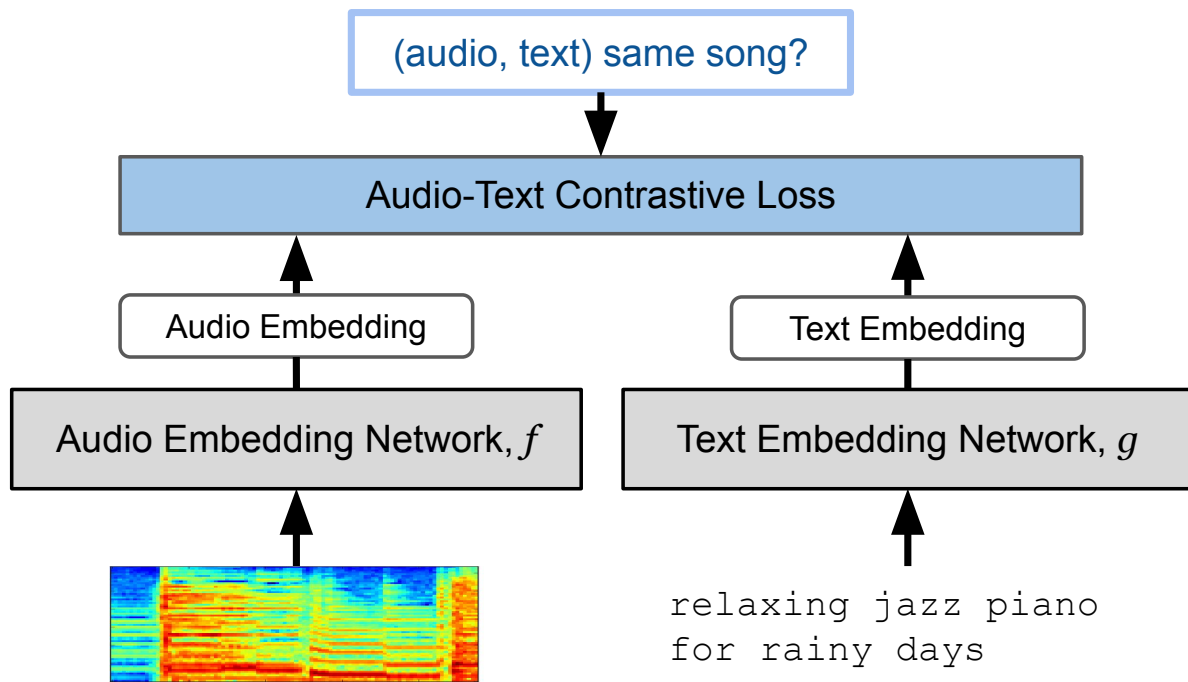


Figure 2.10.: Learning framework of MuLAN [35]

relation. This means that the same sentence could be said by different speakers with different pitches, styles and conditions.

To overcome these challenges AudioLM used two types of audio tokens:- one was the semantic tokens from w2v-BERT [37] and another were acoustic tokens from a SoundStream neural codec [38]. The semantic tokens capture local dependencies like phonetics in speech or melody in piano; as well as long term dependencies like language syntax and semantic content in speech, and rhythm and harmony in piano. These were extracted after downsampling the audio signal as this would allow for long sequences in original sequence. The issue that was obtained after reconstruction of the audio signals was that these provided with poor fidelity. This is overcome to a certain extent when using the acoustic tokens, which helps in generation of high audio quality and long-term consistency.

Given that AudioLM is an audio-only model, it chains several Transformer models together at every stage. In the first stage, the next token prediction is done on the semantic tokens to model the high-level structure of the audio sequence. The second stage takes as input the entire semantic token sequence and the past coarse acoustic tokens and produces future tokens. This stage models acoustic properties like timbre or speaker identification characteristics. In the third stage, the coarse acoustic tokens are modeled with the fine acoustic model to produce even finer details. This is then passed to the Soundstream decoder for generation of the waveform. This is illustrated in the Figure 2.11. . The datasets used were 40k hours of their internal piano music and about 60k hours of English speech.

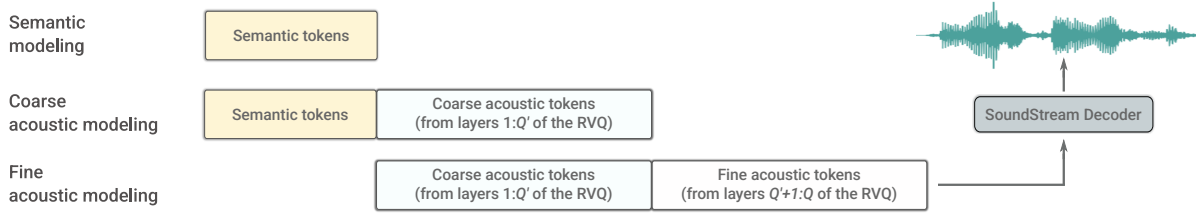


Figure 2.11.: Stagewise token generation and modeling in AudioLM

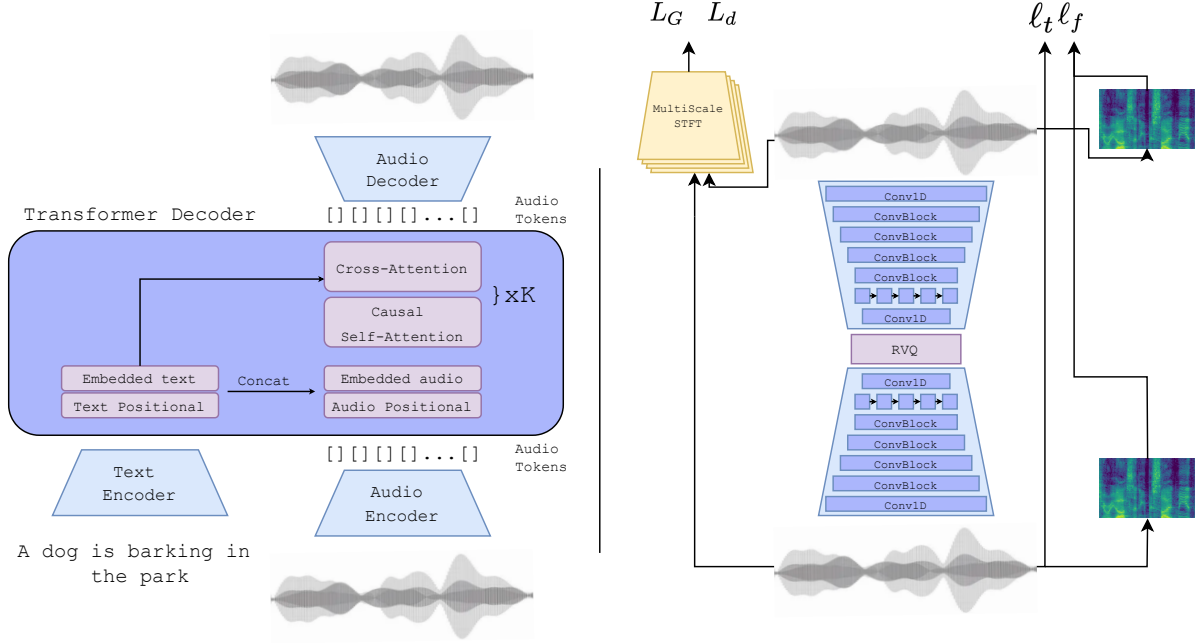


Figure 2.12.: Overview of AudioGen [39]

2.6.5. AudioGen

AudioGen [39] is an auto-regressive generative model that generates audio samples based on text inputs. It also identifies the problems regarding audio representation. Some of the problems are separation of multiple speakers talking simultaneously, background noises, reverberation, etc. The sampling rate for this problem was set to 16kHz. The audio representation model consisted of three components: an encoder network which gets an input audio segment and provides a latent representation as an output, a quantization layer to produce a compressed version of the latent representation using a Vector Quantization layer, a decoder network that reconstructs the time-domain signal from the compressed representation.

Training is performed end-to-end in order to minimize the reconstruction loss that was applied to the time and frequency domain. The overall framework can be seen in Figure 2.12. The objective during training was to optimize a GAN based training objective which would minimize the combination of reconstruction losses and adversarial losses. The text representation is obtained from a pre-trained T5 text-encoder [40]. The authors tried another approach for the text embedding but discarded it later due to the fact that there might be unseen examples

that were missed during training. To obtain better text adherence, cross-attention was added between the audio and text in each attention block of the transformer. The whole system could be viewed as an encoder-decoder structure where the encoder is T5 and is fixed and pre-trained throughout the training. A similar approach was used in this project as well.

Several datasets were used during training. There was around 4k hours of training data that was available. Some data augmentation techniques were also applied where audio samples were mixed together with some noise and the text captions were concatenated together. The evaluation metric used was Frechet Audio Distance (FAD) [41]. The authors found two major limitations to this approach: modelling long range sequences and high inference time.

2.6.6. Riffusion

The base of Riffusion [42] is Stable diffusion. Stable diffusion is a technique which generates images from text. Riffusion is a technique which fine-tuned the diffusion model to generate music by converting the music to spectrograms, and then assuming the spectrograms as images and then using diffusion process on them.

An audio spectrogram is a visual representation of the frequency in an audio sample. It is computed using Short-time Fourier Transform (STFT) which approximates the audio to a mixture of trigonometric waves with varying amplitude and phases. Since STFT is invertible, the original and generated audio could be created from the STFT. In Riffusion, the model only contains the amplitudes. The phases are approximated using Griffin-Lim algorithm. Generation of small pieces was simple with this approach. However, to produce longer clips, image-to-image generations were performed. The interpolation was performed in the latent space of the model.

2.6.7. MusicGen

MusicGen [43] was introduced as a single language model which operates over several streams of compressed music tokens. It has a single transformer model with efficient token interleaving patterns, which would eliminate the need for hierarchical modelling or up-scaling. MusicGen can produce high-quality samples in both, mono and stereo scenarios.

MusicGen used an EnCodec for audio tokenisation. This is a convolutional auto-encoder with a latent space quantized using Residual Vector Quantisation and an adversarial reconstruction loss. The innovation that was provided in this model was the various interleaving patterns of the codebooks.

The modeling conditioning was performed on text as well as on melody. For the textual modeling, experiments were performed using T5 encoder, FLAN-T5 and CLAP. All of these have their own strengths and superiority. For the melody conditioning, it was jointly done via audio's chromagram and the text description. Due to reconstruction of the original signal and hence overfitting in many cases, an information bottleneck was applied by choosing the most dominant time-frequency bin at each step. During training different test preprocessing

techniques were used followed by other ablation studies. The model was trained using a "delay" interleaving pattern which translated 30 seconds of audio to 1500 autoregressive steps. The dataset used for training was around 20k hours of licensed music, of which 10k hours were internal and on Shutterstock and Pond5 music data collection which had 25k and 365k instrument-only music tracks. These were sampled at 32 kHz and the metadata had textual description and information like genre, BPM and other tags which were also used for the textual representations.

2.6.8. Moûsai

Moûsai [44] was developed with an intention to bridge text and music via a text-to-music generation model which would be fluent, expressive and had the capability to handle long-term structure. Thus the Moûsai model was created where there was cascading of two latent diffusion models that generates multiple minutes of high-quality music at 48 kHz from text.

The two stages involve a diffusion magnitude autoencoder (DMAE) to compress the audio waveform using a diffusion autoencoder, and a latent text-to-audio diffusion model to generate a latent space while conditioning on text embeddings obtained from a frozen transformer language model. In the first stage, the audio is converted to a magnitude spectrogram and then encoded into the latent space. Additionally as a part of data augmentation, a small amount of noise is added to the original audio and the 1D U-Net [paper] is trained to remove the noise. This provides a noiseless audio for the next steps. The autoencoder that was used was compression like in an image diffusion model and then to further compress and enhance the representation space by diffusion-based autoencoding. Another critical module is the efficient 1D U-Net, which consisted of only 1D convolutional kernels as opposed to 2D kernels which was more efficient and faster. The 1D U-Net designed consisted of more modern convolutional blocks, attention blocks, conditioning blocks and improved skip connections while maintaining the hourglass architecture.

For the text conditioning a pretrained frozen T5 language model was used to generate the text embeddings. Further classifier-free guidance (CFG) with a learned mask of probability 0.1 was used to improve the strength of text-embedding during inference. Figure 2.13 shows the overall architecture of the model. For the training purpose, a new dataset TEXT2MUSIC was created which contained 50K text-music pair which totaled to 2,500 hours. It was ensured that the stereo music was sampled at 48 kHz, covering multiple genres, artists, instruments and provenience. The inference time, memory, real-time factor of Moûsai was lesser than Riffusion. It also produced an improved CLAP score than Riffusion.

2.6.9. Mustango

Mustango [45] was a music-domain-knowledge-inspired text-to-music system whose base was diffusion. It aims to control the generated music by the use of general text captions and also information related to musical instructions like chords, beats, tempo and key. At the core, it is a

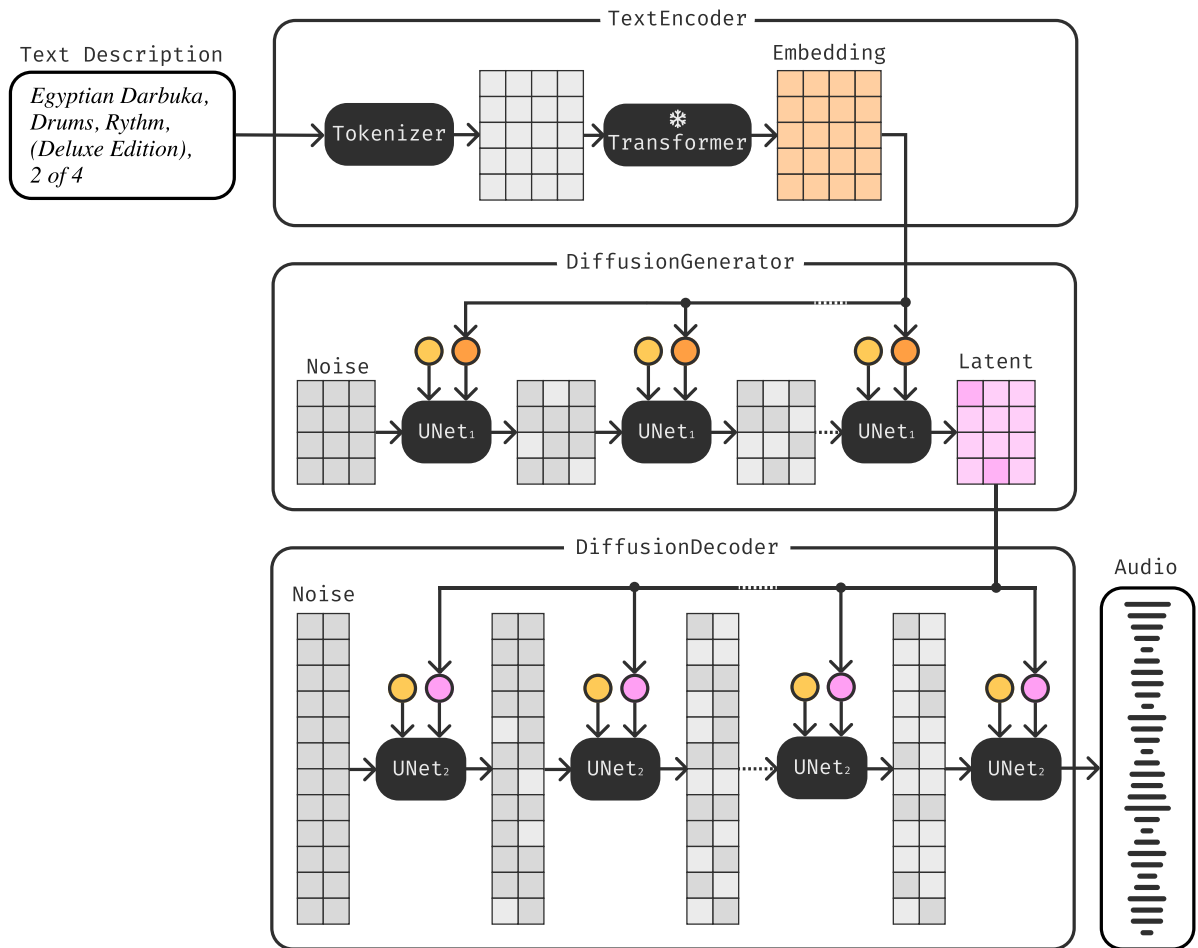


Figure 2.13.: The overall architectural plan for Moûsai

music-domain-knowledge-informed UNet guidance module which allows the model to generate music with emphasis on the music-specified conditions guided by the text prompt with the general text embedding during reverse diffusion process. Due to lack of open datasets of music with text captions, a novel approach to acquire more data which includes altering the harmonics, rhythmic and dynamic aspects of music. SOTA Music Information Retrieval methods to extract music features were used and appended to the text description. Thus, MusicBench dataset was created which contains over 52K instances of music and music-theory-based descriptions in the text. It was also seen that it outperforms other models like MusicGen and AudioLDM2.

Four common music feature: beats and downbeats, chords, keys, and tempo, were extracted from the music and used to enhance the text prompts. BeatNet was used to extract the beat and downbeat features. The second feature tempo was estimated by averaging the reciprocal of the time interval between beats. Chordino was used to extract the chord features. Essentia's KeyExtractor was used to extract the key. These features were then expressed in a text format in different templates.

The data augmentation process was different from the other SOTA processes. The individual samples were altered in either pitch, speed or volume to alter the melodic, rhythmic, or dynamic aspect of music. PyRubberband was used to shift the pitch of the music within a range of ± 3 semitones such that not too much pitch shifts are there to change the natural timbre. The speed was also changed to $\pm(5 \text{ to } 25) \%$. The volume was also altered using gradual volume change from 0.1 to 0.5 times the original amplitude. To generate more paraphrased text prompts, ChatGPT was used.

Mustango builds on its predecessor Tango and uses the similar Latent Diffusion Model. However, during the reverse process of denoising MuNet is proposed. The multi-head attention was applied to the text, the beats and the chords. The MuNet consists of a UNet [paper] and multiple cross-attention blocks. Two encoder blocks were used for beats and chord features which provide Musical Positional Encoding. The Figure 2.14 describes the Mustango model in its entirety. During the training phase, teacher forcing was used so that the ground truth beats and chord features were used to condition the music generation process. However, during the inference process, two transformer-based text-to-music-feature generators that were trained independently were used to predict the beat and chord features. For beat prediction, DeBERTa Language Model was used which takes the text caption as input and predicts the beat count of the corresponding music and the sequence of interval duration between the beats. FLAN-T5 Language Model was used as a chords predictor which takes concatenated text and verbalized beats as input and generates verbalized chords sequence with timestamps as an output.

A few limitations that this work had were limitations to the western music as controllability might be missing for other types of music types. It also just produces music to upto 10 seconds due to its computational constraints. It does not produce long-form of music.

2.6.10. M²UGen

M²UGen [46] is a multimodal model which draws inspiration from pretrained models like MERT, ViT and ViViT for music, image and video. For generation of music, it uses Audi-

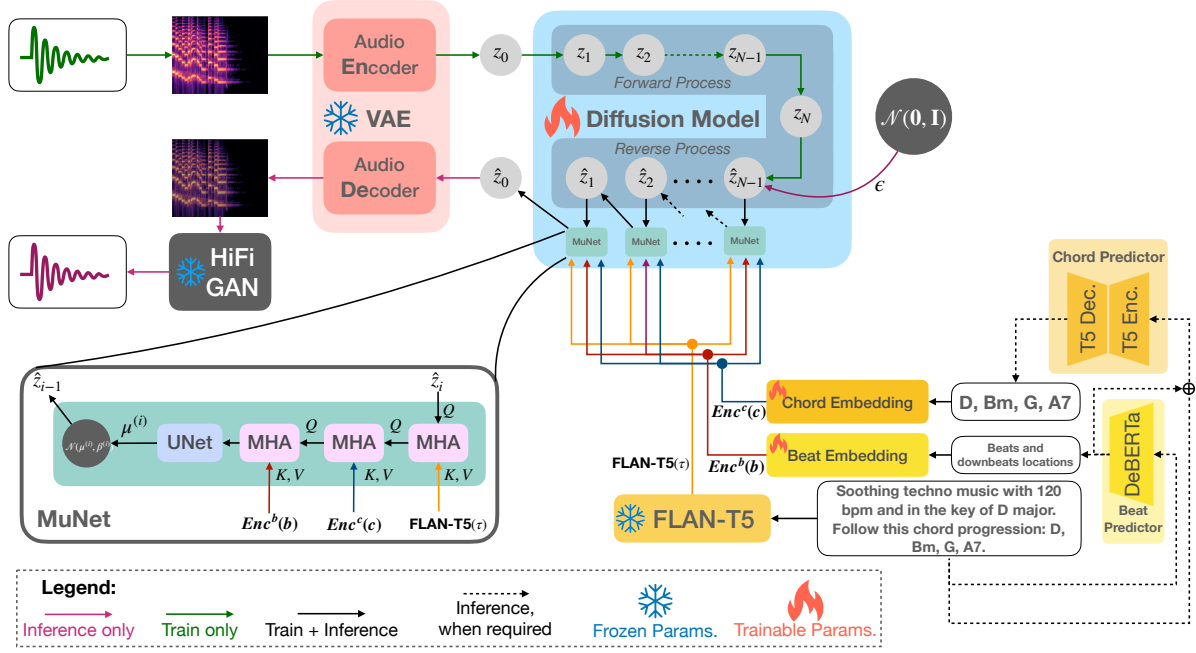


Figure 2.14.: Mustango's overall architecture

oLDM2 and MusicGen. The integration was accomplished through LLaMA 2 model. For music encoding, MERT was used which performed well in music tagging tasks as opposed to Wav2CLIP and Jukebox. MERT model generated feature embedding for the music inputs. The shape of the output embedding is (25, 1024) which is obtained by stacking the 4 hidden layer and the final output layer of the MERT model. In the case of image encoding, ViT was used as it has promising breakthrough due to its performance and image encoder. The shape of the output of ViT was (197, 768) where 197 is the number of 16×16 patched in a 224×224 input image along with the final output layer output. 768 is the hidden size of the transformer. For video encoding, ViViT was used which produces an embedding of (3137, 768) where 3137 is the total count of 16×16 patches sampled uniformly from 32 frames of size 224×224 , including the final output layer. For the integration of all these embeddings, a multi-modal understanding adapter was built. The output of the adapters were then fed to specific layers of the hidden layers of the LLaMA 2 model. The output of the LLaMA model was used as a module for generation of the music understanding and generation tasks.

For the training, due to very high number of parameters, most of the models were frozen and LoRA fine-tuning approach was used. The dataset used was a mixture of a lot of different datasets which had different modalities to be mapped to audio. Authors proposed in the future to enhance the model's fine-grained music understanding capabilities and improve the correlation between the generated music and input instructions. The overall architecture is seen in Figure 2.15.

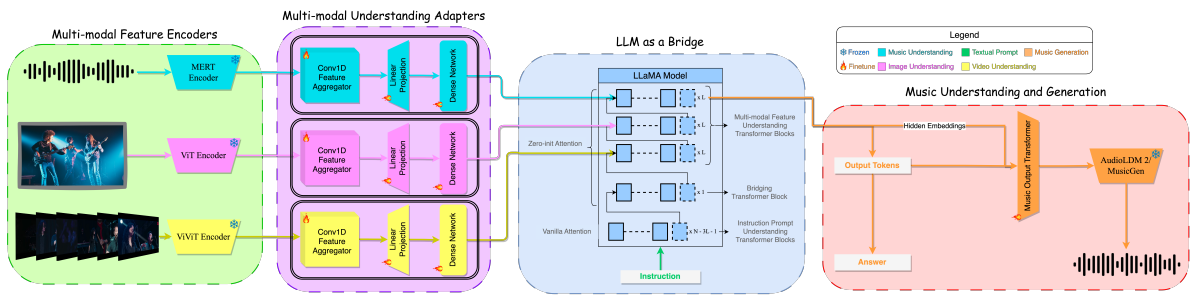
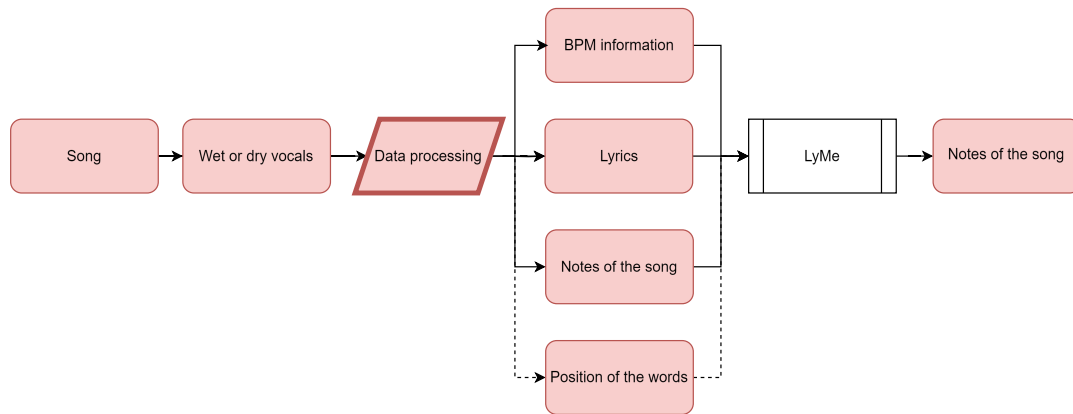


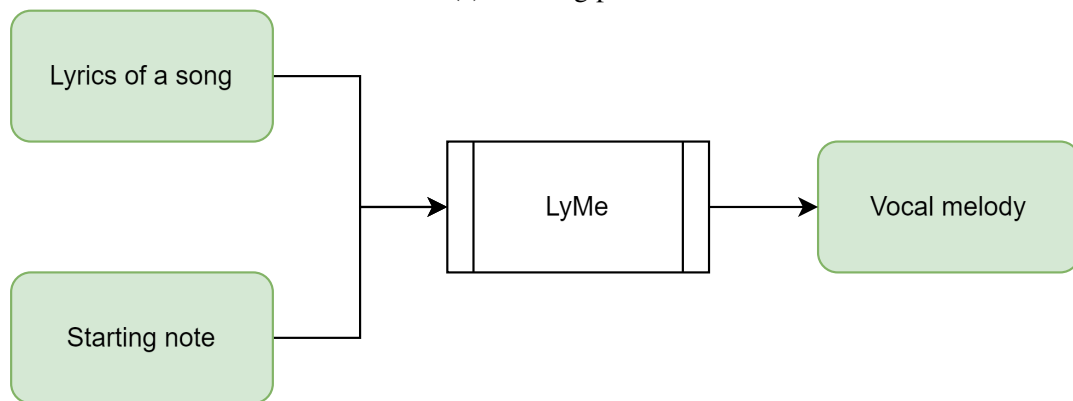
Figure 2.15.: Overall architecture of M2U Gen

3. Proposed algorithm

Chapter 1 introduces LyMe. Figure 3.1 shows the training and inference part of LyMe. This chapter covers the various algorithms that are used for the data processing part during training, along with their relevance and mathematical expressions wherever possible. During the data processing, it is important to identify information on the vocal notes of the song, lyrics and BPM of the song. Pitch estimation and VAD are essential in identification of the notes. The ways to determine the notes, BPM and lyrics are present in this chapter. It also contains several comparative studies between the algorithms. Information on different models are also included in this chapter.



(a) Training part



(b) Inference part

Figure 3.1.: Training and inference part of LyME

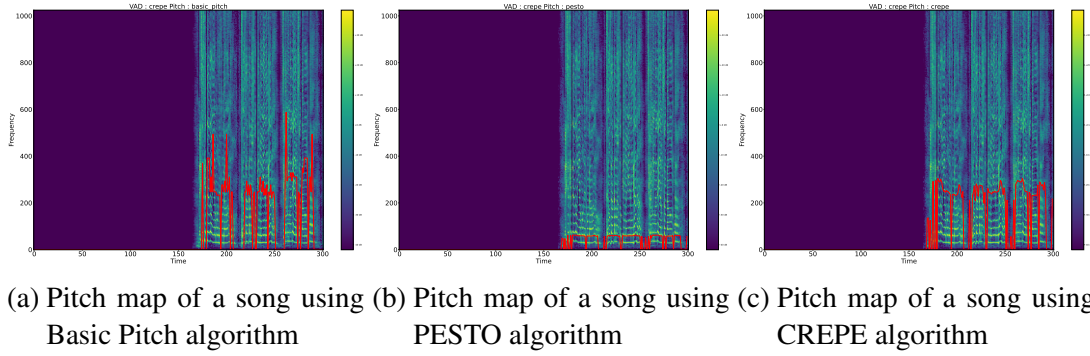


Figure 3.2.: Comparison of pitch detection algorithms for a partial song

3.1. Pitch detection for Singing Voice Melody Extraction

This section discusses the various pitch detection algorithms that were discussed in Section 2.1. This is required for the generation of the melody part of the objective. The pitches were taken of vocals only, since the focus is on generation of the vocal melody. The algorithms CREPE [7], PESTO [8] and Basic Pitch [9] were applied to various vocals and their results were noted. The algorithms provide with the frequency and other information for every 100 ms interval. The frequency of all the algorithms were plotted to see the most favourable one to be used. In Figure 3.2, the graph regarding the algorithms with the spectrogram of 30 seconds of a song is shown. We applied all the three algorithms and analysed the outcome of the pitch detected by them. From the Figure 3.2, PESTO can be eliminated easily as the pitch detected was very low. Comparing CREPE and Basic Pitch, it was found that Basic Pitch has a tendency to produce higher pitch notes in many scenarios. Though CREPE was not perfect at all scenarios, it was closest to the fundamental frequency of the vocal songs.

On further analysis it can be said that the reason for CREPE to perform better than PESTO is because it is a supervised learning approach and the results from [8] also shows that CREPE performs better than PESTO. CREPE was trained in a supervised manner and has 750 times more parameters than PESTO whereas PESTO was trained using a SSL based approach on the same datasets. Basic Pitch was trained on 22050 Hz and mainly trained on instruments rather than on voices or vocals. Hence the performance of Basic Pitch for pitch prediction of vocals is not suitable and produces errors. The same trend could also be seen in various other songs. Some of them are shown in Appendix A.2.

Another issue with Basic Pitch algorithm was that it used to detect the background noises in the vocal data. The model has a normalisation layer at the start which normalises the data in a given section for which the pitch is to be predicted. If there is some noise in the data which is in the range of -30 dB or lower, then it detects the noise as an actual audio and detects the pitch. In order to negate this effect, the audio was first passed through a filter to remove any artifacts which is lower than -30 dB. Then the algorithm produced reasonable output but the performance of CREPE was better than Basic Pitch as can be seen in Figure 3.2.

Thus, CREPE was the algorithm that was decided to be used for the pitch detection.

3.2. VAD algorithms - comparison

Similar to the pitch detection algorithms, there was a need to compare the VAD algorithms. One might question the need for a VAD since the pitch detection algorithms give the details of the pitch. It was seen that CREPE and PESTO produce pitch even when there is no audio. The only factor that changes is the confidence by which it predicts this. The confidence that the model gives as an output becomes low when the model thinks that there is no sound, as can be seen in Figure 3.3. This is one of the ways with which the VAD can be detected. However, their performances varied with different situations. Based on the analysis of 10 random songs, the following issues with each of the VAD algorithms mentioned in Section 2.2 were made by hearing the actual vocals and on applying VAD algorithms:

1. CREPE (Root-Mean Square (RMS) of the confidence)

- It detects sound where no sound present.

2. PESTO (RMS of the confidence)

- It misses some parts when singing for a long time continuously.
- It also has an issue with running for longer time durations of non-silence.

3. Basic Pitch

- Some parts of the vocals are ignored.
- Starts of the vocals are not prominently detected.
- On-set threshold and frame-threshold are two parameters that need to be tuned per song and can not be set constant for every song. However, for the purpose of this thesis, they were kept constant to their default values.

4. Silero

- It is very prone to noises and provided information on the places where there is silence.
- It has yodeling and vibrato issues as well as multispeaker issues.
- In some instances, produces high pitches when there is a break in the song.

5. Pyannote

- It gives more information. It is prone to noises.
- There are issues with yodeling and vibrato. It does not detect them as it assumes that they are noises.
- There are issues with multiple speakers. It changes midway from one speaker to another.
- In some instances, produces high pitches when there is a break in the song.

- It stops before the end of the songs in some cases.

Based on the analysis, it was found that RMS of CREPE or PESTO works the best. It is a simple algorithm and produces the best results out of the more complicated procedures. An example of using CREPE as the pitch detection algorithm and also as VAD can be found in the Figure 3.3. It shows that the pitch detection is not perfect and at many instances not correct but overall it provides a good estimate of the pitch.

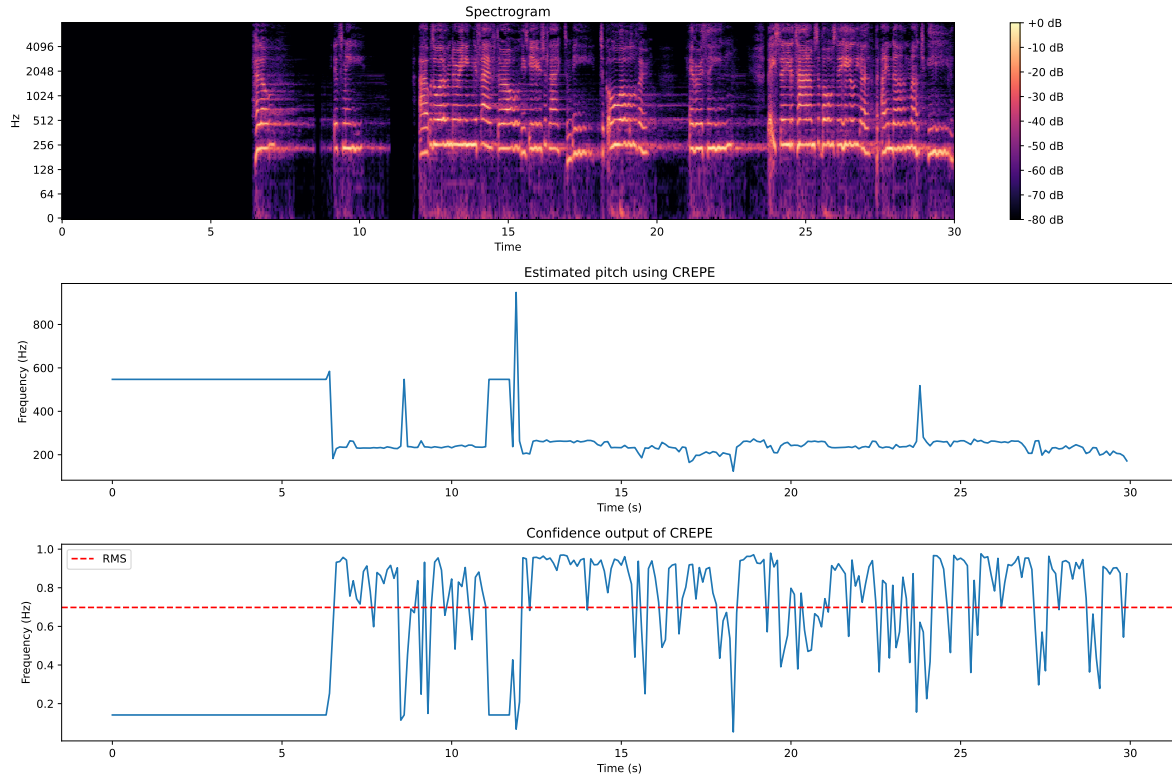


Figure 3.3.: An example to show the actual spectrogram, the pitch as found by CREPE, and the confidence of a voice as identified by CREPE algorithm with the RMS value marked in red

3.3. Audio-to-text with word timestamping

Whisper [26] has multiple models from which to choose from. For the purpose of this project, the "medium.en" model was used, since the songs that were being used were all English songs from various genre and had various timestamps. The songs had different sampling frequencies. In order to be consistent, and for the pitch detection algorithms and the VAD algorithms to work effectively, it was necessary to resample them to same frequency. For this purpose, the base frequency of 16 kHz was chosen. The audio was then converted from stereo to a

mono. This was done by taking the average of both the channels in stereo output. Whisper's "medium.en" model was used to find out the text in that audio waveform.

In the later part of the thesis, there was a need to know which word was being "sung". For this, training data was needed. Whisper has a parameter "word_timestamps" which had to be enabled. It provides the words along with the start and end times within the audio segment. Thus, there is a need to take all the segments and then provide the combined word timestamping.

3.4. Pitch shifter and BPM

Given that the songs could be in any key, it was necessary to "normalize" the songs such that the transformer could gain more information as it would understand that all the songs are "normalized" and would have a certain range. The "normalization" process was carried out sample wise. For each sample, the most frequently occurring musical note was considered and it was mapped to C4.¹ The amount of semitones was measured in order to change the most frequent note to C4 and accordingly the whole song was shifted by the same amount of semitones. This helped in maintaining the song's musicality and correlation to the shift in pitches or notes.

Another aspect to consider was the BPM. The *librosa* library was used. The function of beats and tempo provides the approximate BPM of the sample. It operates by first computing a time-frequency representation of the audio using a technique like STFT. Then, it analyzes the spectrogram to detect rhythmic patterns to indicate the tempo. It is often estimated by identifying peaks in a tempo-gram, a two-dimensional representation of tempo over time. The function returns an estimate of the dominant tempo in BPM based on the detected rhythmic structure of the audio signal. The BPM used in our case is the average BPM of the sample.

3.5. Positional encoding algorithms

One important aspect of the transformer is the way the positional encoding is performed. There are four of them as mentioned in Section 2.5. Out of them, the linear is very rarely used due to the issues mentioned and the inherent positional encoding is already present in the BERT encodings. The other two, namely sinusoidal positional encoding and rotary positional encoding, were coded and are displayed.

3.5.1. Sinusoidal positional encoding

The theory and the algorithm behind sinusoidal positional encoding was discussed in Section 2.5.3. The output encoding is added to the data to provide the information about the position.

¹It is important to note that this step is not musically valid as the most frequent note need not necessarily be the key of the song.

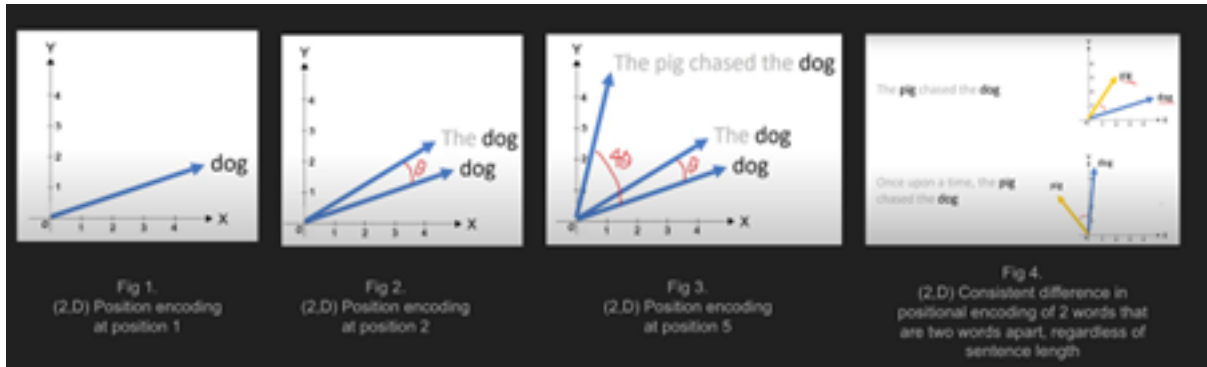


Figure 3.4.: An example of RoPE positional embedding taken from [47]

This encoding can be added to either the output embedding of the BERT, the embedding of the notes or the embedding of the position of the words. More information on how to apply this and the algorithms followed will be discussed in Section 3.6.

3.5.2. Rotary positional encoding

Similar to sinusoidal positional encoding, the theory and the algorithm was discussed in Section 2.5.4. The output of the algorithm can be directly be used as the information about the position is already embedded into the input. This encoding can be performed on the input embedding of the BERT, the positional embedding of the notes or the embedding of the position of the words. More information on how to apply this and the algorithms followed will be discussed in Section 3.6.

An example of RoPE could be seen in Figure 3.4. This shows the real-world example and it shows that the even though the ground truth position of the words are different, the difference in the position is constant. This is what can be encountered in RoPE but not in the sinusoidal position encoding.

The mathematical derivation of the algorithm is present in Appendix A.1. The Algorithm 2 is the algorithmic description of it. The output of this is provided to the transformer as it already contains the positional information with respect to the tokens and the features.

3.6. Data

There were two types of data used:

1. Flute dataset - The files corresponding to the flute dataset was extracted from LAKH dataset [48]. The LAKH dataset contained MIDI files corresponding to different music instruments and other information and metadata. However, since a monophonic tune was needed to be similar to the requirements of the task, flute melody was selected. Flute melody produces monophonic sounds which is ideal for this task. The MIDI

instrument number that flute corresponded to was 73. Data from other wind instruments like clarinet, trumpet, trombone etc could also be used.

2. Internal dataset - The data for getting the lyrics and corresponding vocal music was obtained internally. This composed of two parts - dry and wet. The dry component of the vocals did not contain any audio effects like reverberation, etc. The wet component contained artifacts. There were around 1800 wet songs which consisted of around 22 hours of lead vocals, around 60 hours of lead vocals male, and 33 hours of lead vocals female. In the lead vocals part, it can be either male or female.
In the case of dry songs, there are two different datasets that are present. They have a combined total of 75 hours of dry music available for usage. The first dataset, dataset A, which was obtained by modification of the previous wet data. This dataset composed of around 18 hours of songs. The second dry dataset, Dataset B, was much larger and had around 57 hours of dry songs.

The flute dataset is without the words or lyrics and hence could be used only for understanding music. It would help the model to understand different aspects of music like placement of notes, association between notes, BPM and its correlation with notes, and other information. The use of the two dry datasets in the music-and-lyrics was to solve the main issue at hand and to understand if the model performs good only on certain types of songs or can it generalise on any kind of music which has lyrics associated with it. The songs in these datasets are different, have different genre and styles. This would help the model not overfit on one type of music but generalize over different variety of music.

3.6.1. Data processing

The processing of the data took place in several steps:

- The songs were of various sampling frequencies. The flute dataset had frequency of 22,050 Hz whereas the frequency of the internal dataset was 16 kHz. The data was resampled down to 16 kHz. This was done using torchaudio's resample method.
- Since we are working in a one-channel approach, it is necessary to convert the songs which had two channels to one channel. The left channel and the right channel were averaged to get the desired audio.
- The songs could have varying lengths. Thus for uniformity, the length of each data sample was set to 20 seconds. So, for example, a song with 5 minutes of song would have 15 samples of 20 seconds each. Each of these samples were then divided into 0.1 seconds interval. This was done because the resolution needed by the pitch detection algorithms was 100 milliseconds. Hence, each sample consisted of 200 datapoints, each representing 100 milliseconds of the song.
- Each 20 second sample was passed through the pitch determination algorithm CREPE to determine the pitch in terms of the frequency (Hz). The same sample is passed through the VAD algorithm, also CREPE, to provide the region of voice activity. Since the VAD

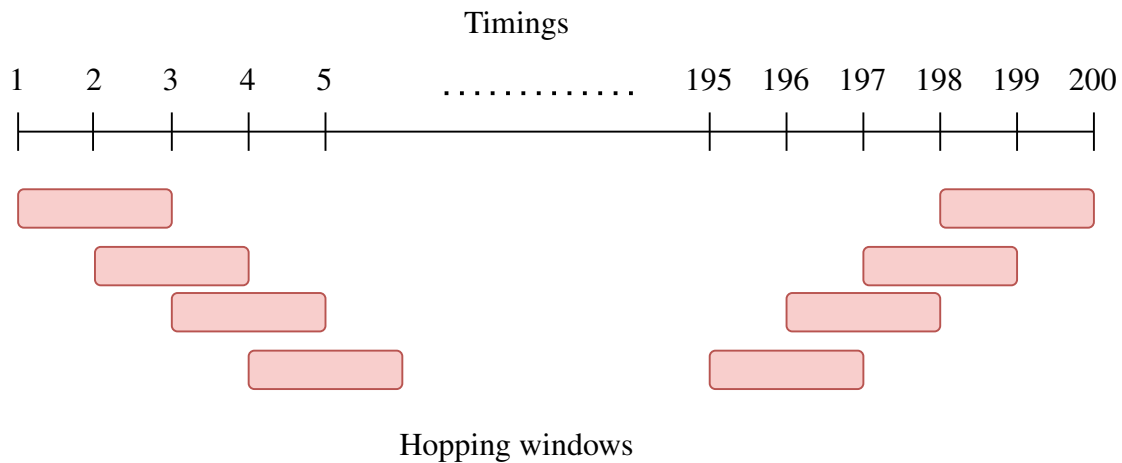


Figure 3.5.: An example of the hopping window where the hop is 1 sample and the total length of the window is 2 samples. The timing is in 100ms scale. In our dataset, the hopping window was 2 seconds or 20 samples. 20 samples represent 0.2 seconds.

output is 0 – 1, 0 for no voice activity, and 1 for voice activity, it is multiplied with the frequency or pitch. This gave the total frequency response of the vocal music in Hz.

- During the initial processing, it was seen that the amount of silence in the songs were very high. This led to the problem that the model would always emphasise on the silence token and not output any of the other notes. A song usually starts and ends with instruments. Since we are working with vocals, there would be no vocal activity towards the start and end of the song. This could also be present in smaller quantities when there is a change in verse or the singer pauses before singing the next part of the lyrics.
- To diminish the effect of silences, the amount of consecutive silence was limited to 2 seconds. This had two significant effects: one, it reduces the number of data samples, and secondly it helped in balancing the amount of silence tokens with the other notes.
- To compensate for the reduced number of samples, hopping windows were introduced. It divides the data into overlapping windows of a fixed sized and each window is treated as a separate sample. Figure 3.5 shows an example of hopping windows. An example to show how this helped to increase the number of datapoints would be that the LAKH dataset after removal of the silence portions from the individual samples only gave 50,831 datapoints. After the introduction of the hopping window of 2 seconds, it resulted in 506,162 datapoints. So, the increase of data was approximately 10 times.
- The next part of data processing would be converting the pitch in frequency (Hz) to their corresponding pitch in music scales. The frequency-to-note conversion could be seen in Table A.2. However, a problem with this is that they are discrete and not a continuous, whereas the pitch estimation is continuous. They were converted to the nearest notes by calculating the nearest frequency and therefore the corresponding note. This created an issue with estimating vibrato. The pitch changes quickly and subtly, which alters the

pitch estimation algorithm's output. This impacts the ground truth that was to be given as an input to the model and hence the output.

- Given that the scales of the samples could be anything and in order to provide more useful information to the model, a pitch shifter algorithm was applied. It is detailed in Algorithm 3. It is to be noted that this algorithm of pitch shifter is not musically correct. As the amount of data obtained from the dry datasets is less and too much varied across all the notes, this step is an attempt to lessen the variation by centering the data around a note, C4 in our case.
- After the shift, the 20 second sample is passed through Whisper to get the text. From here, two information is extracted: the overall text information and also the timestamps of each word activity. This gives another crucial information for the model.
- Another data that is also extracted from each of these samples is the BPM information using *librosa.beat.tempo*.
- After the timestamped word is obtained from the Whisper model, it is then passed through another algorithm which provides the position of each word corresponding to the pitch. The algorithm is present in Algorithm 4. The output *word_timestamped* consists of 3 columns: time (to indicate the timestamp in the sample), words (to show which word is being played), position (the position of the word to indicate the length till which the word is to be played). In case at a given time instance no word is being played, a "Silence" is inserted in the words and the position of the word is increased by one. In doing so, one issue that occurred is that due to the erroneous nature of the pitch and Whisper algorithms, an original sentence of 4 word like "I love you boy." is being mapped to "0,0,1,1,1,0,0,2,2,2,0,0,3,3,3,0,0,0,4,4,0,5,5,...". The 5 that occurs cannot be mapped back to the words as there are only 4 words. This is due the fact that the pitch detection algorithm or the VAD detection algorithm detected a silence or a non-voice component. This led to an issue. To resolve this, one method would be to output a 1 when a new word is detected. So the above would be mapped to "0,0,1,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,...". This corresponds that the word "I" started at 0.2 second, "love" started at 0.7 seconds and so on. This is much simpler and can give the model more information and also flexibility.

We performed these data processing steps and the final dataset contained information of 20 seconds of a song with the pitch information in frequency domain, in scale or note domain, the word position of the words detected and also the BPM of the sample. Each of these were used in the model at different algorithms and different stages.

Algorithm 3 Pitch shifter**Input:** Input tensor $pitch_vad_info = [x_1, x_2, \dots, x_n]$ **Output:** Output tensor $pitch_shifted = [y_{0_1}, y_{0_2}, \dots, y_{0_n}]$ **Output:** Output tensor $notes_shifted = [y_{1_1}, y_{1_2}, \dots, y_{1_n}]$ $notes = get_notes(pitch_vad_info)$ ▷ Provides notes from pitch algo and VAD $scales = get_notes(notes)$ ▷ Provides scales from notes like C from C4 $note_to_midi_no = midi_convert(scales)$ ▷ Converts scales to MIDI no - Table 3.1 $most_freq_scale = get_most_freq_scale(note_to_midi_no)$ ▷ Most frequent scale $shift = 60 - most_freq_scale$ ▷ Scale song so most freq note - C $notes_shifted, freq_shifted = shift_scale(notes, shift)$ ▷ Shifted notes, shifted pitch info**Algorithm 4** Word Position estimator**Input:** Input : $word_transcribed$ from Whisper**Output:** Output : dataframe $word_timestamped = [word, position_info]$ $all_segments = get_all_segments(word_transcribed)$ ▷ Collect word segments from Whisper $expand_df = replicate_words(all_segments)$ ▷ Duplicate the start and end times at an interval of 0.1 seconds to obtain 200 datapoints $word_timestamped = fill_word_pos(expand_df)$ ▷ Given the words and timestamps, fill in the positions consecutively or as 0-1.**3.6.2. Models and data associated with them**

As described in Section 2.4, a transformer with text conditioning would be used. The encoder of the transformer would be a BERT encoder and the decoder would be conditioned to generate the melody or the notes. Further, it would also be able to generate the word positions for the user to understand which word has to be sung when. However, between the encoder and the decoder, there is a linear layer which maps the output of the BERT model which reduces the number of features of BERT from 768 to 256. This significantly reduces the number of parameters to around 10% of the original. After this, different models with some parametric changes and some data changes were tried to see the effect of these changes on the outcome. The model had the output of the BERT provide an input in their decoder layer. After this, there were positional encoders for the encoder output and the decoder inputs. This followed the decoder where 4 decoder blocks were used with 8 attention heads and 128 neurons in the feed-forward layer of the block. This was followed by dropout layer with a probability of 0.1 and terminated by another linear neural layer to map the 128 to the number of features provided by the BERT model. The whole model terminated with a feed-forward layer which mapped the output of the decoder to the 110 classes of the musical notes.

110 classes of musical notes consisted of a silence token, 108 notes as present in Table A.2 and a token for end-of-song. Initially, the output of the model is the probability of the note

Scale	MIDI Number
C	60
C#	61
D	62
D#	63
E	64
F	65
F#	66
G	67
G#	68
A	69
A#	70
B	71
B#	72
0	0

Table 3.1.: Scale to MIDI Number Mapping

being present. Further, the model was changed to include the word positions also. Thus, the final model has the potential to output two tokens at a given instance of time: one token for the note and another for determining the word position to understand which word would be sung when. The decoder was fed the word positions in the 0-1 mode as well in their absolute form. For this, another layer was added along with the output layer of the decoder which could determine the position also as a classification problem.

The way the models were developed are as follows:

1. Base model - This model contained the text and the corresponding melody. This did not use the full capacity of BERT. BERT provides two outputs - one is the encoding of the text and the other is the attention mask for the encoding. Initially, it was assumed that the model would be smart enough to accommodate for the attention mask without actually providing the attention mask to the model.
2. Base model with padding - The second model that was trained was incorporating the attention mask of the BERT. This was fed to the transformer as the mask for the memory. This improves the output of the model as it provides more information to the model

about the tokens and their usage.

3. Base model with padding and attention maps as loss function - We saw that the second model performed better than the base model. In there, the losses were calculated from the cross-entropy loss of the notes at the output. When the cross-attention maps of the decoder were plotted, it was seen that the cross-attention was also attending to the padded part of the text.

An example of the padded part of the text would be, if the text is "I am", the BERT tokenizer with the maximum length set to 200 gives 200 tokens when padding was set to "*max_length*" - one for start, one for 'i', one for 'am', one for end-of-sentence and the rest would be filled with 0s. The reason for setting the padding to "*max_length*" to maintain uniformity in the amount of tokens being provided to the model. Since different length of sentences would have different number of tokens given as an output of BERT, this padding was done. Here, if the sentence provided too many number of tokens, then they were truncated.

In the cross-attention maps of the decoder, there was significant attention being provided to 0s of the BERT which would be useless as they have no meaning in the real world. The most amount of energy of these attention maps should be in the non-zero part of tokens of BERT. The idea of this came from [49]. In order to concentrate the whole attention on the non-padded part of the text, a regression loss was also incorporated along with the cross-entropy loss of the model. The regression loss is calculated by the Algorithm 5.

Algorithm 5 Attention loss

Input: Input : *attention_maps* from model

Output: Output : *loss*

$avg_energy \leftarrow 0$

for i in *attention_maps* : **do**

$non_padded_energy \leftarrow \sum_{j=1}^{sum(attention_mask)} i[j]$

$total_energy \leftarrow \sum_{j=1}^{length(i)} i[j]$

$ratio_of_energy \leftarrow \frac{non_padded_energy}{total_energy}$

$avg_energy \leftarrow avg_energy + ratio_of_energy$

end

$avg_energy \leftarrow \frac{avg_energy}{4}$

$loss \leftarrow 1 - avg_energy$

4. Base model with absolute word positioning - Till the previous models, it was not possible for giving the melody to a singer to sing as the correlation between the words and the melody would not be present. The timings for the words would not be present. Thus a model was created in which the absolute word positioning was introduced in the model. This meant that the model would have to be changed to incorporate two inputs to the

decoder: one for the notes and the other for the word position. This meant that there is a requirement for another embedding layer, and another positional encoding/embedding layer. This word positioning could use a different decoder. However, then it would have to learnt two things and might have seen them differently as different sequences and learnt them independently. Hence, the same decoder was used twice: once for predicting the notes and the second time for the prediction of the position of the words. We did this as this should enable the model to learn the correlation between the word position and the notes that the model provides as an output.

5. Base model with 0-1 word positioning - Given the disadvantages of absolute word positioning as discussed in Section 3.6.1, the word position was changed to a binary classification.
6. Base model with special character separation - Given that the encoder is only dependent on BERT and BERT itself has various rules of breaking words, it might be difficult for the model to understand if the tokens correspond to one word or not. An example would be "[120.909bpm] I express unhappiness" which in BERT language is converted to ['[CLS]', '[', '120', '.', '90', '##9', '##b', '##pm', ']', 'i', 'express', 'un', '##ha', '##pp', '##iness', '[SEP]', '[PAD]', '[PAD]', '[PAD]', '[PAD]'] when the max_length is set to 20 and the model used is 'bert-base-uncased'. This creates a problem as this would indicate that there are more words and the separation is an issue. To solve this, a separator of '#' is introduced in place of ' '. After this, the input text becomes "[120.909bpm]#I#express#unhappiness" and the tokens become ['[CLS]', '[', '120', '.', '90', '##9', '##b', '##pm', ']', '#', 'i', '#', 'express', '#', 'un', '##ha', '##pp', '##iness', '[SEP]', '[PAD]']. There are more tokens but there is a clear demarcation between the words. This information may be useful to the model and could help in the correlation between the notes and the text.
7. Base model with special character separation and 0-1 word positioning - This is a mixture of previous 2 models.
8. Base model with special character separation and 0-1 word positioning and wet dataset - In this model, the dataset that was used to train the model is the wet dataset and the validation was done on dataset B of the dry dataset.

Various types of results and their interpretations are listed in Chapter 4. Model 5 and 6 can be seen as an ablation study of model 7. The architecture of the model is present in Figure 3.6. For some of the models like model number 1,2 and 3, the dotted lines were not used, as the model did not need word position as an output. In the other models, word position was an output of the model. So these dotted lines became solid lines are a part of the model.

In a real-world scenario, the dry datasets would not be available. The provision of getting the wet data would be to use source separation techniques to extract the vocal part of the music [50]. There is a use of BS-Transformer which would help in separation of vocals from songs. After the vocals are extracted, the data processing as mentioned in Section 3.6.1 would be done. This would give the dataset on which the model 8 could be used. The flow diagram for the same can be found in Figure 3.7.

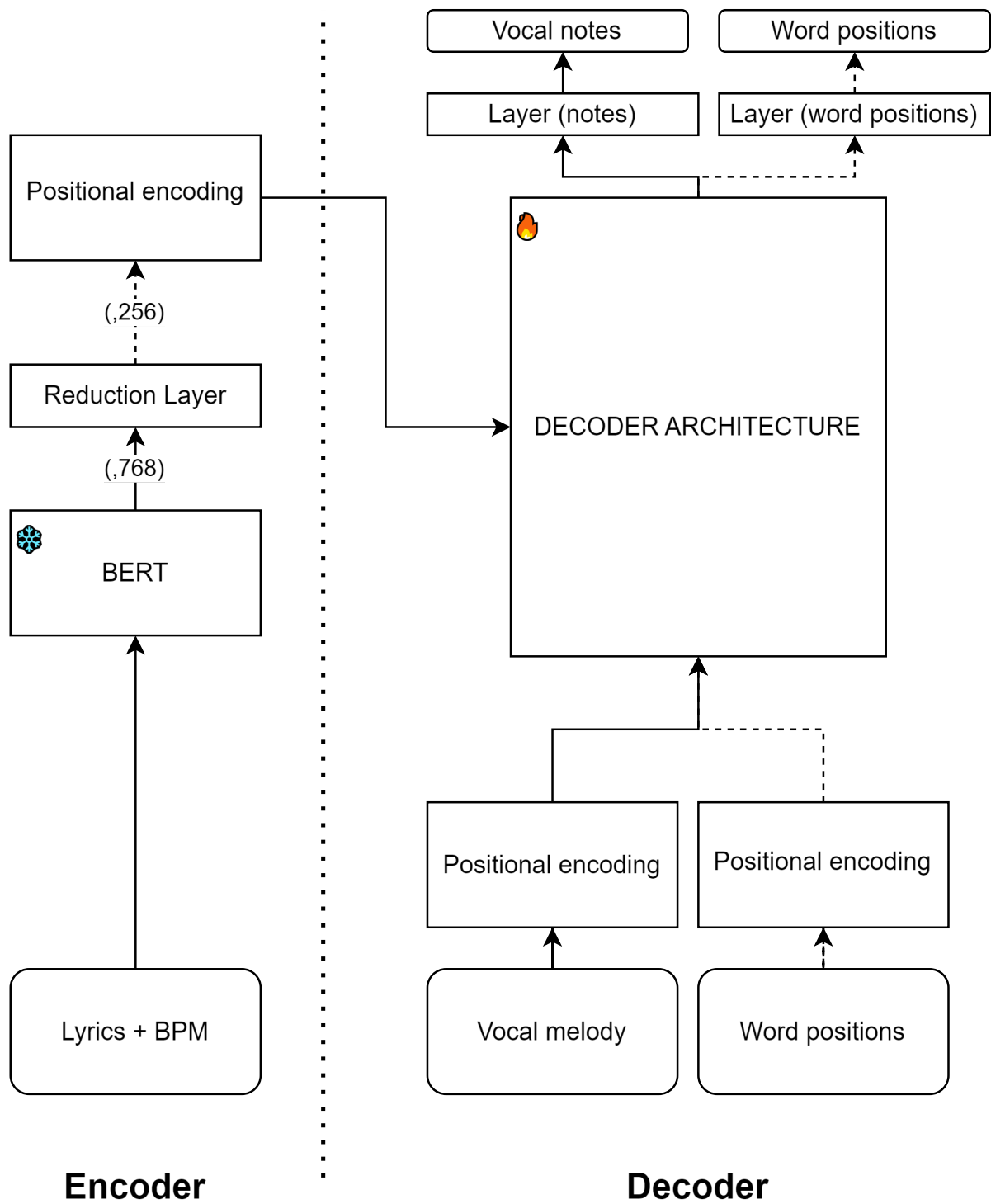


Figure 3.6.: Overall model that was used. The dotted arrows show that in some models or configurations they were used whereas in others they were not used. The generation that is being done is autoregressive in nature.

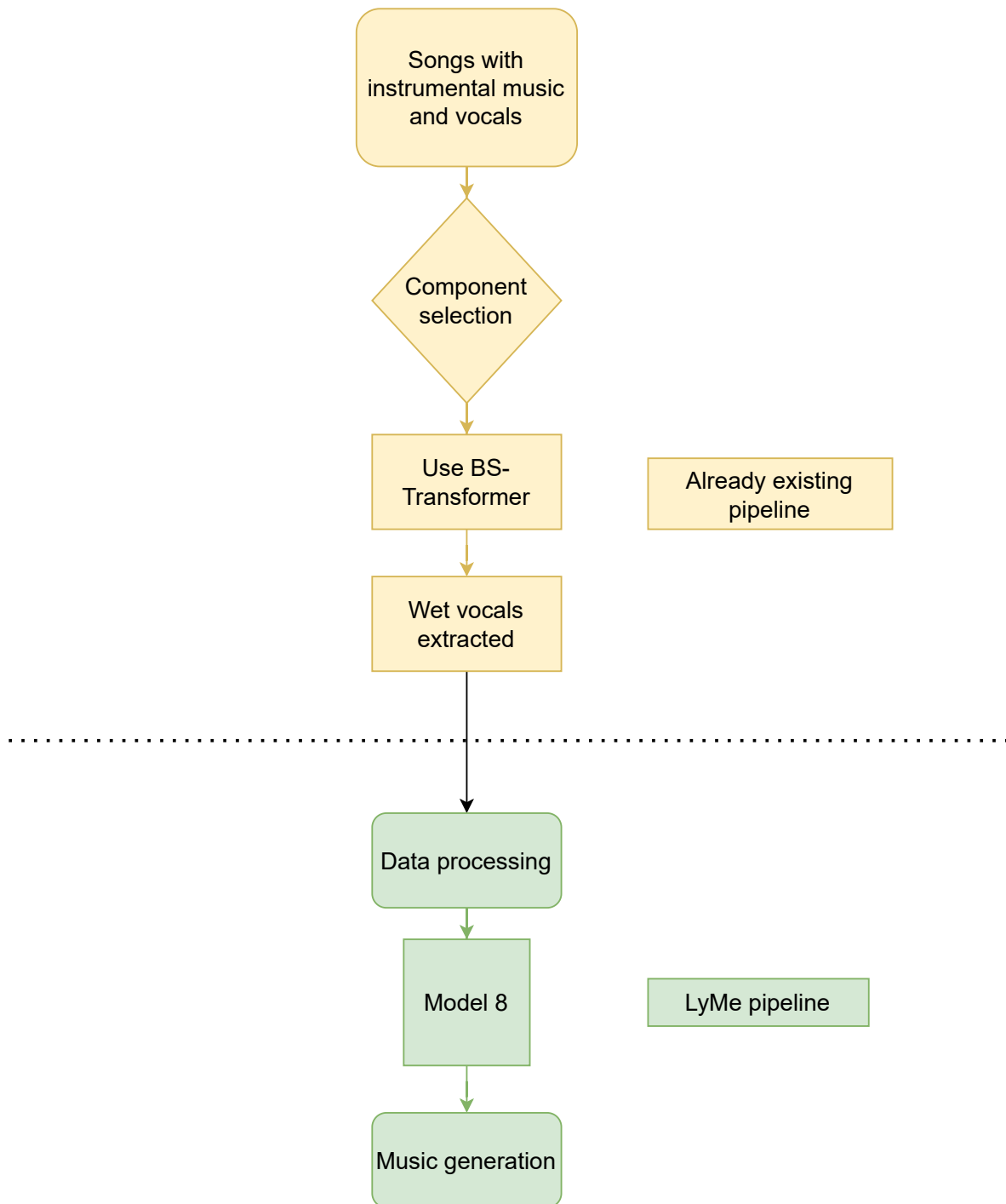


Figure 3.7.: End-to-end pipeline for training LyMe and generating vocal melody. We have assumed that the left side of the pipeline had already been implemented and the data needed for the right side of the pipeline is available. In our case, the available datasets were the wet and the dry datasets. The right side of the pipeline has been implemented as a part of this thesis.

4. Results

The comparison of the different pitch detection algorithms and VAD algorithms were discussed in Section 3.1 and 3.2. We concluded from those experiments that we can use CREPE as the pitch detection algorithm and also for the VAD. This chapter focuses on the results that the models gave and also on graphical analysis of them.

4.1. Models

Most of the models have some parameters which have been kept constant for the purpose of the project. They can also be fine-tuned in the future. They are :

- BERT model - 'bert-base-uncased'
- Number of attention heads - 8
- Reduced BERT - 256 [BERT provides a mapping to 768 features. Using 768 features to the transformer was creating 19.8 million trainable parameters. However, if the feature mapping was reduced to 256 by using a linear layer, the number of parameters reduced to 2.6 millions]
- Learning Rate - $1e-3$
- Learning Rate update - Multiplied by 0.5 after every 50 epochs
- Number of decoder layers - 4
- Dimension of the feedforward neural network inside the decoder - 128
- Optimiser - AdamW with β s as (0.9, 0.98) and ϵ as $1e-9$
- Loss function - Cross Entropy loss

These can be changed, and they can further be tuned to achieve other results. At many instances the transformers were warmed up. This approach stabilizes optimization by preventing large fluctuations in the early stages. It also avoids divergence by easing the model into higher learning rates, and encourages the model to explore the solution space for faster convergence and generalization. To achieve warm up, Algorithm 6 was followed with the linear rise of the learning rate from 0 to the learning rate of the model in 3 steps.

An important thing to note is that the accuracy is calculated as the amount of correct notes

predicted given a note. Since the length of each input sequence of notes is 200, then the maximum accuracy that any model can achieve is 199 as the starting note has to be given to the model.

Algorithm 6 Warmup Training Algorithm

Input: Input : data from the training dataset

Input: Input : learning rate lr

for i in $[0, ..., lr]$: **do**

$count = 0$

while $count < 1000$ **do**

 Run though the model with sampling data from training dataset

 Compute losses

 Update the weights using backpropagation

$count = count + 1$

end

end

4.1.1. Base model

The base model was created to understand if the transformer was capable of handling this task or not. The basic structure of the transformer was used. The model summary is provided in Appendix A.3. The total number of parameters are 2,628,462 or roughly 2.6 million. For training this model, the dry dataset A and the LAKH dataset was used. After the data processing step, the training data had 416,568 samples whereas the validation data had 14,995 samples. It is to be noted here that the LAKH dataset does not have any textual information. It only contains the BPM information. The model would look like :

```
SimpleTransformerDecoderModelSmallModel(
  (fc_in): Linear(in_features=768, out_features=256, bias=True)
  (embedding): Embedding(110, 256)
  (pos_encoder_0): PositionalEncoding(
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (pos_encoder_1): PositionalEncoding(
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (transformer_decoder): TransformerDecoder(
    (layers): ModuleList(
      (0-3): 4 x TransformerDecoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(
            in_features=256, out_features=256, bias=True)
          )
      )
    )
  )
)
```

```

(multihead_attn): MultiheadAttention(
  (out_proj): NonDynamicallyQuantizableLinear(
    in_features=256, out_features=256, bias=True)
)
(linear1): Linear(in_features=256, out_features=128, bias=True)
(dropout): Dropout(p=0.1, inplace=False)
(linear2): Linear(in_features=128, out_features=256, bias=True)
(norm1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
(norm2): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
(norm3): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
(dropout1): Dropout(p=0.1, inplace=False)
(dropout2): Dropout(p=0.1, inplace=False)
(dropout3): Dropout(p=0.1, inplace=False)
)
)
)
(fc_out): Linear(in_features=256, out_features=110, bias=True)
)

```

There would be 3 positional encoders that were used in this thesis: learnt one (the output of BERT would not be subject to any positional encoding), sinusoidal and rotary. For this model, sinusoidal and rotary were applied to both, notes and word embeddings of the BERT, whereas the BERT's positional embedding was only applied to the memory element which is the word embeddings of the BERT. From Figure 4.1, it can be seen that with or without warm starting the transformer, the accuracy is best achieved when the positional encoding of BERT is used for the positional encoding of the words. The positional encoding for the notes is RoPE in this scenario. The same configuration but using sinusoidal positional encoding for notes is present in the Appendix A.4.

It was concluded that for this model and dataset, the BERT positional encoder was performing the best in both the training and validation datasets. It was also concluded that the warm start of the transformer was helping in reducing the initial loss, but during the long run, the training loss were almost similar to each other, and the validation loss was also comparable. One such example is present in Figure 4.2. The same pattern was found for the other configurations as well. Another factor that is to be seen was the the distribution of the notes during the validation step. It is illustrated in Figure 4.3. It is very evident that the frequency of the 0th class which is the "silence" class is very high as compared to the other classes. If that class is ignored then the next most sampled class is 24 which is represented by C4. Hence, it can be concluded that the validation data follows the similar pattern as the training pattern. This was due to the fact that each sample was "normalized" such that the most frequent note in each sample was mapped to C4 and the other notes were scaled accordingly.¹

The training was performed in a teacher forcing manner whereas the validation was done in a teacher enforcing way as well as by using the argmax of the probability of the classes involved.

¹This is not musically correct, but was done to simplify the problem and give the model some background.

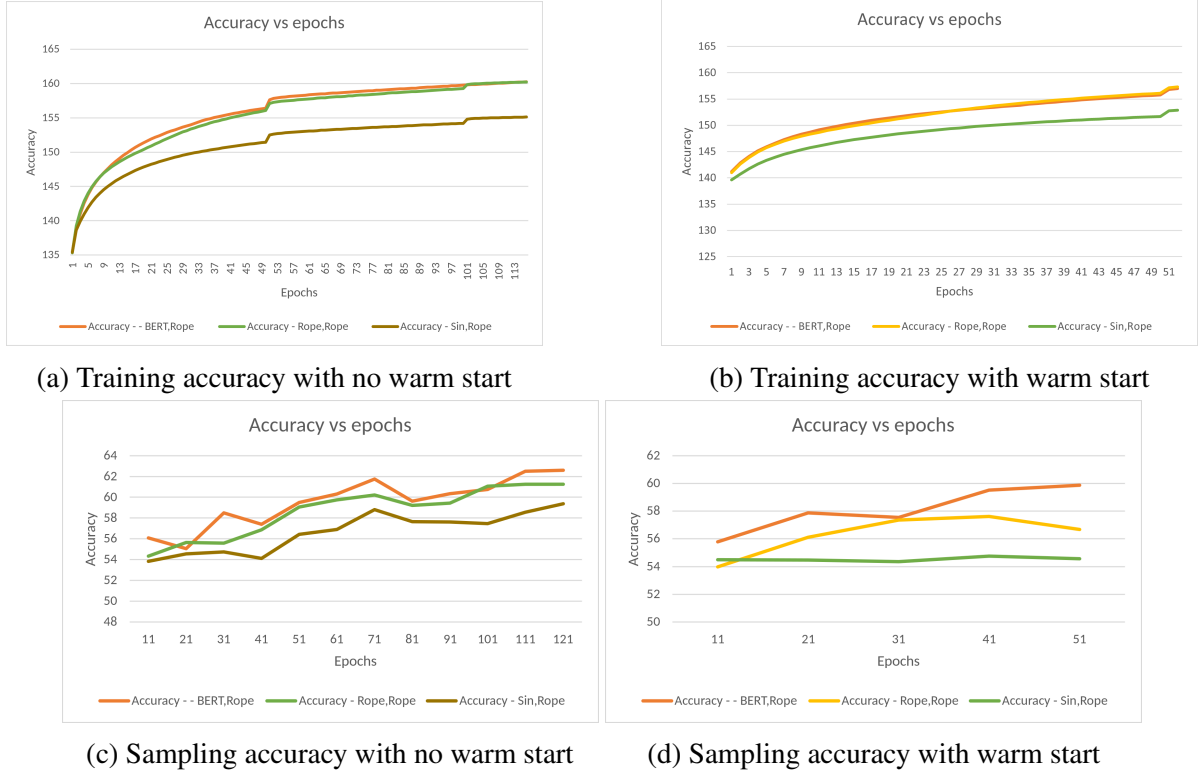


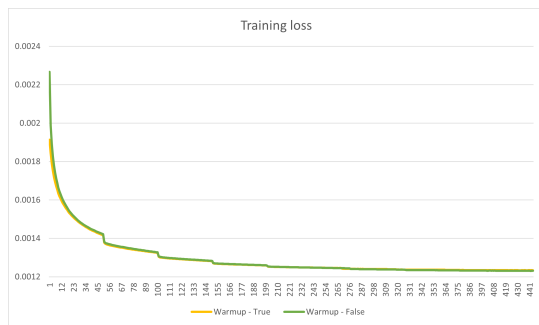
Figure 4.1.: Base model training and validation graphs

The results of the argmax approach has been shown. Teacher forcing uses the ground truth from the training dataset at the current time step $y(t)$ as input in the next time step $X(t+1)$, rather than the output generated by the network. In our case, the ground truth is the notes.

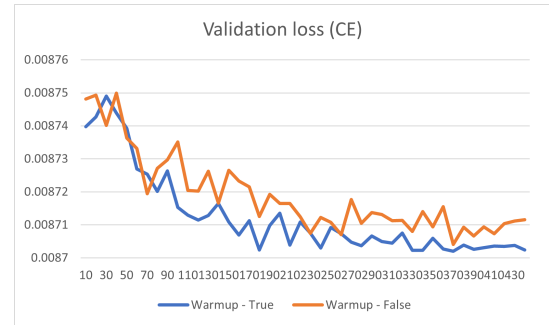
4.1.2. Base model with padding information

As described in Section 3.6.2, the second model uses the padding information that BERT provides. This was passed to the decoder layer as a mask to the BERT encoding output, such that the model understands to focus only on certain tokens and not on all of the input. It was assumed that this would enhance the model's performance in terms of accuracy of the validation and significantly reduce the loss. The dry datasets A and B were used for this as the use of BERT would be needed. Without the use of the padding information the accuracy of the training dataset plateaued at 138.7 whereas for the validation set the maximum accuracy was achieved at 66.82, with the trend line showing that it would plateau at 68.25. This was seen when there was no warmup of the transformer, the positional encoding of the notes was RoPE and the positional encoding for the words was taken from BERT. The dataset that provided with this was the dry datasets A and B.

However, in the case where padding information was added to the transformer, there were few changes that were observed. The figures for the same are presented in Figure 4.4. It was seen



(a) Training loss with the notes given RoPE encoding and the words being provided with sinusoidal encoding



(b) Validation loss with the notes given RoPE encoding and the words being proved with sinusoidal encoding

Figure 4.2.: Base model loss graphs for the training and validation with change in the warmup of the transformer.

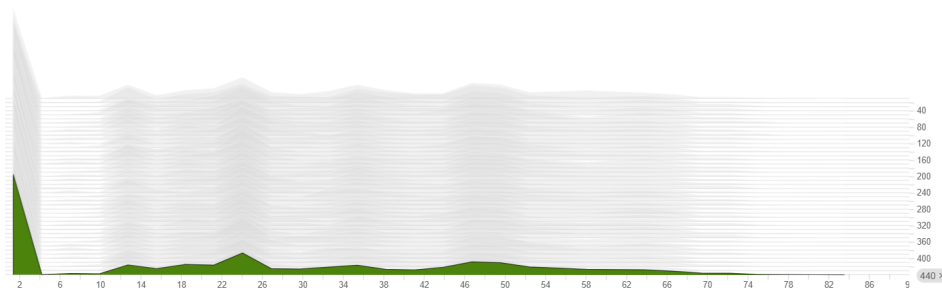


Figure 4.3.: Note distributions where X-axis is the class of the notes and Y-axis represents the frequency of note distribution. The different timestamps or epochs are also represented on the vertical right hand.

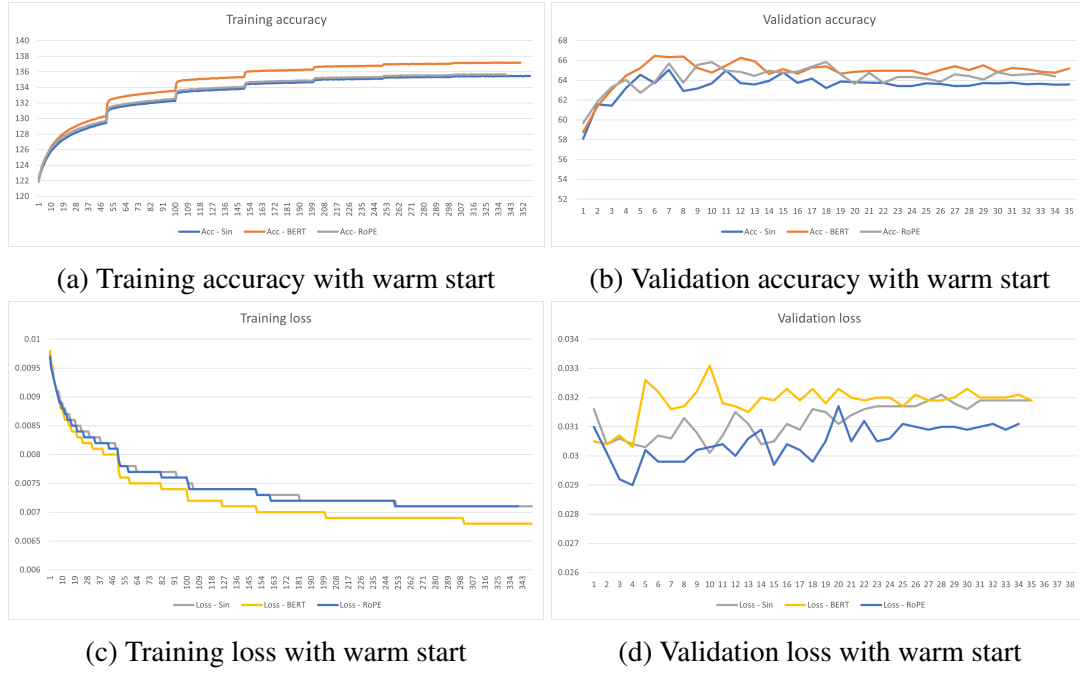


Figure 4.4.: Base model with padding - training and validation graphs. The transformer was warmed up and sinusoidal positional encoding was applied to the notes

in this scenario that when learnt embeddings were used, we got better results in the training. However, the validation loss of RoPE encoding when applied to the word embedding was lowest amongst all three with comparable validation accuracy. In this case the positional encoding of the melody or notes were set to sinusoidal. Similar study was done with RoPE being used as a positional encoding for the notes with varying the positional encoding of the words in Appendix A.5.

The use of padding information was crucial as it significantly improved the results. It could also be seen from Figure 4.4 that the model starts to overfit. It is evident from the fact that the validation loss had started to increase with the decrease in the training loss. The major reason for this is the unavailability of larger, well-labelled data.

It can be seen in the Figure 4.5 that there is improvement in terms of accuracy in the validation dataset when padding is used. Thus for all the models padding has been used. The configuration is that the positional encoding for word embedding and note embedding is RoPE.

4.1.3. Base model with padding information and additional loss function

The model was using cross entropy loss of the ground truth of the notes and the probability predicted by the model. Another loss function that we used was the loss derived from the cross-attention maps. The cross-attention heads would attend to the words while querying the melody. Thus the cross-attention maps would be generated. However, it has been seen that

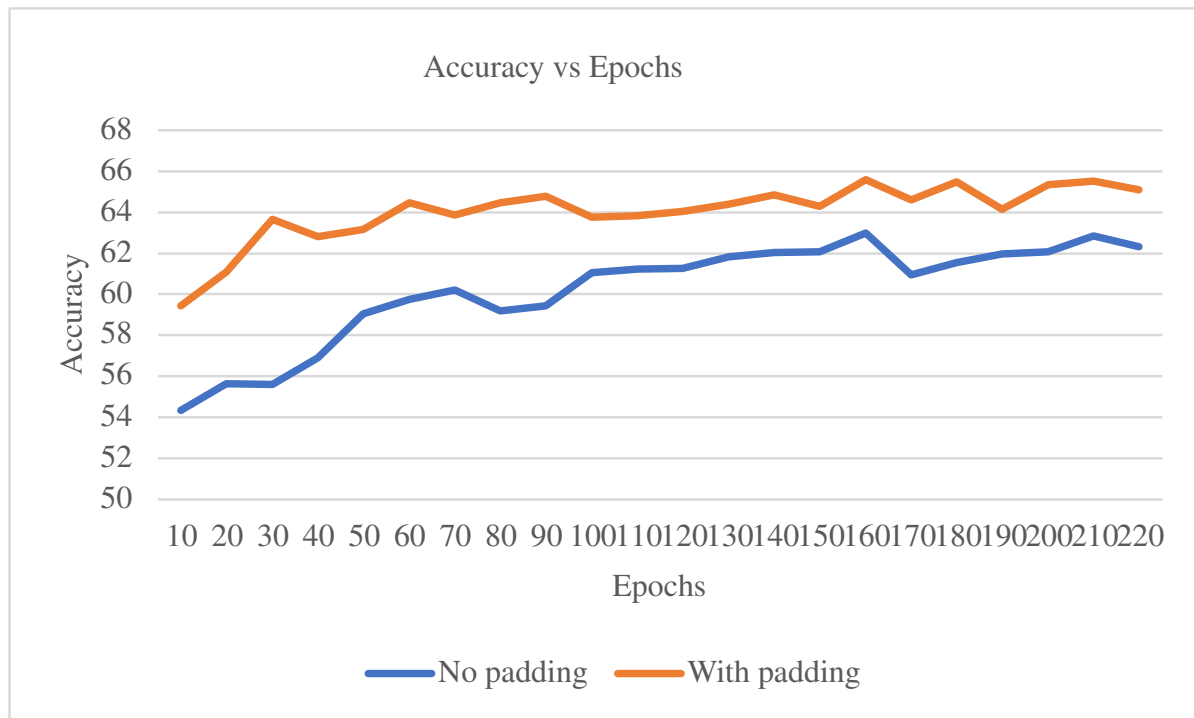


Figure 4.5.: Base model with padding vs base model without padding. The validation accuracy over time increases 4 notes. The accuracy calculated here is the total number of correct notes predicted by the model. The maximum value that the accuracy can have is 199.

the cross-attention maps attend to the padded BERT tokens and provide substantial weights to them. In order for the maximum energy of the cross-attention maps to be included only in the actual BERT tokens and not in the padded tokens, a regressive loss was used. Here, the average of all the attention maps were taken and the ratio of energy in the cross-attention maps which attended to the actual words to the ratio of energy of the overall was calculated. The loss is one minus this ratio. This could also be seen in the Algorithm 5. This would force the model to try and maximize the ratio, thereby only attending to the actual words and not the other tokens. The loss is a mean squared error loss.

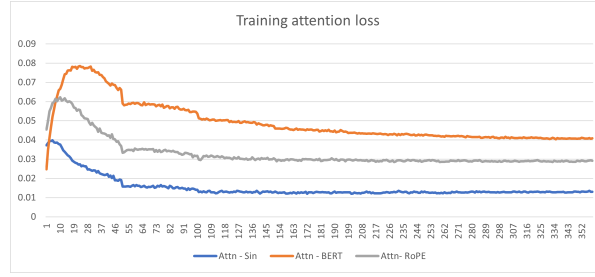
It is necessary for the model to reduce the loss occurred by the melody or note prediction as well as increase the energy concentration in the attention maps on the actual words. Figure 4.6 shows that even though the cross entropy loss for BERT is the lowest, the cross-attention loss is the highest. This signifies that the notes that are generated are not because of the words in the BERT but because of the padding that happens, which is not desirable. The cross-entropy loss of sinusoidal and RoPE encoding when applied to the word embeddings are comparable, but the sinusoidal positional encoding produces less attention loss when used along with RoPE encoding of the notes. Similar trend can also be noticed in when different positional encodings are applied to the words when sinusoidal encoding is used for notes. These can be seen in Appendix A.6.

This also shows that the model has overfit and has achieved memorisation capacity. It also means that the model was heavily penalised for some wrong tasks. The reason for it is that the model, during training, has found a way to rearrange the cross-attention weights such that the loss during training is minimised. However, since these memorizations are not global, they fail during the time of validation. One way to avoid this is to apply a mask on the cross-attention which would specify that the notes should attend to only the word tokens and not the padded tokens. Since this mask is local and not global, the problem of overfitting would be avoided. Another reason would also be the quality and quantity of data. It was seen that when using the LAKH dataset for the first model, there was no overfitting. Hence, the model has the capability to learn to produce melody, but it needs better and larger datasets.

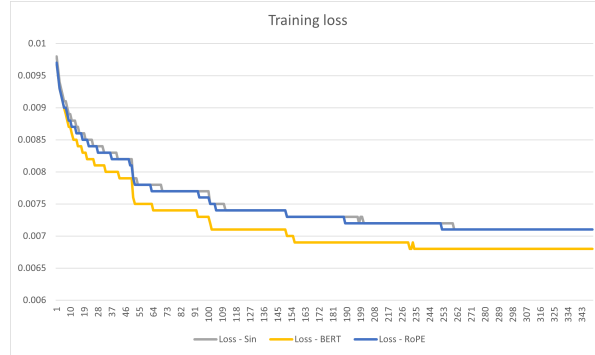
4.1.4. Base model with word positioning

Given the problem statement to generate melody from lyrics, it is imperative that it be known which words in the lyrics are being "sung" at which time instant. Hence, there is a need to create a word position information along with the lyrics generation information. We performed the timestamping of the words using the output received from Whisper. It was evident from the ground truth points that this was not coinciding with the output of the pitch detection algorithm and VAD algorithm. However, this was important to be done and was therefore carried out.

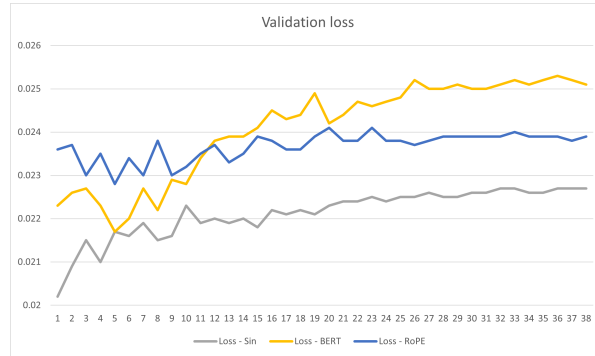
Here, the model would produce two outputs: the notes and the word position. The word position is also defined to be a classification problem and not considered as a regression problem. This helped in simplification of the model and the problem at hand. This problem can be solved in two ways: the same transformer model could be used to provide both the outputs given different inputs, or different transformer decoder would be used to cater to different



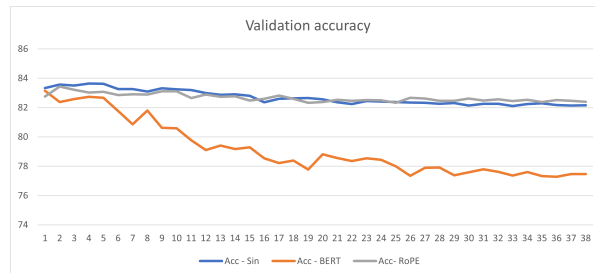
(a) Attention loss during training



(b) Cross entropy loss during training



(c) Total validation loss



(d) Validation accuracy

Figure 4.6.: Base model with padding and attention loss - training and validation graphs. The positional encoding of the notes was RoPE and the transformer was warmed up before training.

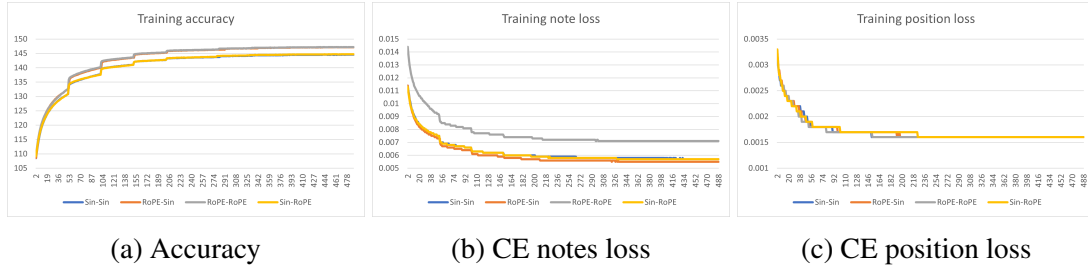


Figure 4.7.: Base model with padding and word position information - training graphs. CE - Cross Entropy. BERT was used as positional encoding for words. The model was warmed up before training.

problems. The second one has a big disadvantage is that the transformers would have no interaction between the word position and the note position and as a result would not be able to find any pattern between them. This is an advantage for the first model. The transformer would have a high change of learning both the notes and word positions independently but one after the other. This would make it understand the correlation between the note positions and the word positions. However, this might also lead to higher loss and eventually overfitting if the ground truth is wrong.

Figure 4.7 shows the execution of the model during training. It can be seen that the accuracy of the model which has melody or notes positional encoding and words' positions' positional encoding to be RoPE-RoPE and RoPE-sin perform better than the others. The embedding for the word tokens is BERT in this scenario and the model was warmed up before execution. On analysing the words' positions' loss graphs, we found that the model has already saturated to some random values which it feels to provide the lowest training loss in terms of position. It can also be seen that the word position loss are similar in nature and this has lower impact on the training accuracy. In the case of note loss, the configuration RoPE-RoPE has the worst loss whereas the others converge to a lower limit. Figure 4.8 shows the validation curves. Some important outcomes can be found after analysis of these. Firstly, the model has overfit. It may be due to the reasons mentioned before. Also, it can be noted that the positional loss is very comparable and hence does not impact the accuracy too much. It can be seen that initially, it was increasing but then it decreased and then remained fairly constant. This is inline with the observation made during the training phase. The model initially overfit and then learns a random mapping which it felt was best and that reduced the position loss. Further, it remained fairly constant. One way to remove overfitting issues and also improve on the positional loss aspect would be to gain access to larger, well-labeled dataset. Another would to have better approach to the word positioning.

4.1.5. Base model with 0-1 word positioning

We found out from the previous model that there were possible data issues with the word positioning and its association with the pitch. We tried another manipulation to the data where the start of the new words are marked. This eliminates problems like the duration of the word

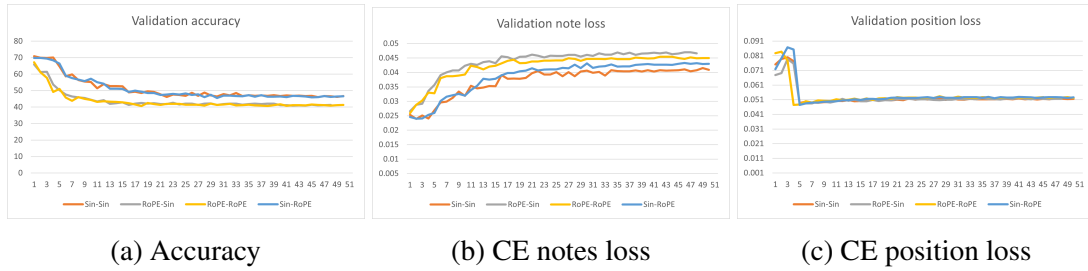


Figure 4.8.: Base model with padding and word position information - validation graphs. CE - Cross Entropy. BERT was used as positional encoding for words.

which can be variable and up to the user or singer. Other problems like the interference of silence tokens which might break the words' progression in the song is also eliminated. Thus the word positioning which previously included the actual positioning of the words is changed to represent the start of the new word by 1. All other instances including silences etc are marked by 0s.

Unlike the linear word positioning, this model does not overfit as the complexity of the data is reduced. The positional loss is very low and understandably so, as the number of positions are only two, 0 and 1. Also the model seems to have plateaued and reached a stable state. These were also seen in Figure 4.9 and Figure 4.10.

When the losses and accuracy are compared to the previous model, it was seen that for the same configuration and dataset, this model performed better. This was due to the fact that the position was either 0 or 1. So, the model was forced to focus more on the notes, thereby achieving higher accuracy on the notes and lower loss. It is important to note that there was imbalance in the number of 0s and 1s in the output. This is because there would be less new words. For example in a text with 100 hundred words would have only 100 1s and rest would be 0s. Singing 100 words in a span of 20 seconds would only be possible if the song was a rap. So, if the number of words are less then the difference between 0s and 1s increase substantially.

Another interesting factor to be seen in the results of the previous model (Figure 4.7 and Figure 4.8) is that around 40 epochs in the positional loss, the model potentially figured out that instead of trying to predict the actual value, a linear rising index might be more suitable and would lead to similar loss. Hence after that epoch, the positional loss is almost constant. This can only be seen in the word's positions' loss with linear word position, not in the 0-1 scenario as the linearity is broken and binarization occurs. This could be seen when comparing the Figure 4.11 with Figure 4.9 for the training phase, and Figure 4.12 with Figure 4.10 for the validation part.

4.1.6. Base model with special character separation

We tried another way to find the correlation between words and notes. We used a user defined delimiter instead of a space between words. This would allow for the BERT model to produce more tokens but this would enable a clear separation between the words and this could be

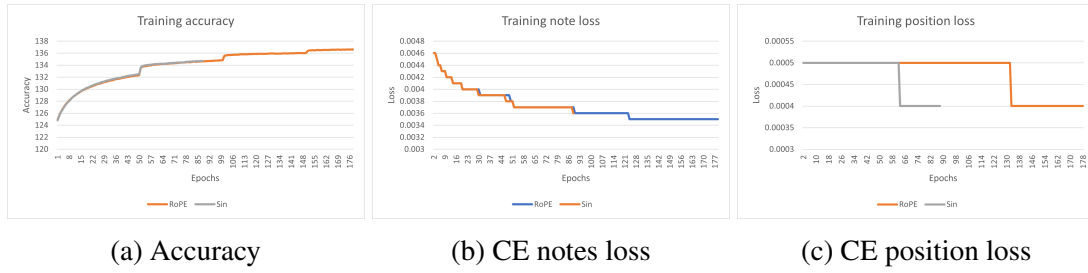


Figure 4.9.: Base model with padding and 0-1 word position information - training graphs. CE - Cross Entropy

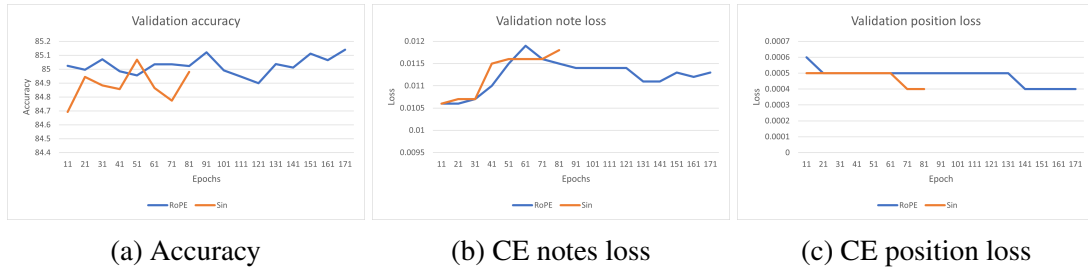


Figure 4.10.: Base model with padding and 0-1 word position information - validation graphs. CE - Cross Entropy

interpreted by the model as a separator and help it understand the correlation between the notes and the words.

The separator used for this purpose is '#'. As expected this produces additional tokens and elongation of the actual tokens by the number of separators. Given that the maximum length of the BERT was 200 and rest of the tokens produced by BERT would be truncated, there were chances that some of them did not get the full text.

This model was trained where the transformer was warmed up. The position of the notes were given either RoPE encoding or sinusoidal encoding. Figure 4.11 and Figure 4.12 also show that the model has overfit. This indicated there is a need to either increase the data or reduce the complexity of the model. The effect of using a user-defined delimiter could be more well-defined if the amount of data was more. Also, another study could be to use more tokens. In the thesis, the maximum number of tokens used was 200. The effect of increasing this might effect the overall performance.

4.1.7. Base model with special character separation and 0-1 word positioning

This model combines the previous two models. The data provided to the model tends to separate the words by the special character '#' and also has the 0-1 word positioning in it. This should immensely help in determining the correlation between the notes and the words, and also help in determining the start and end of the words and their corresponding notes.

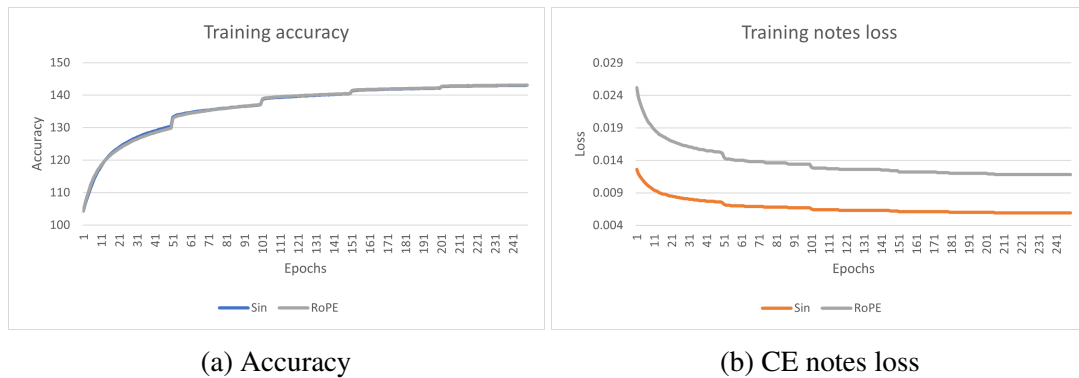


Figure 4.11.: Base model with padding and special character separation - training graphs. CE - Cross Entropy

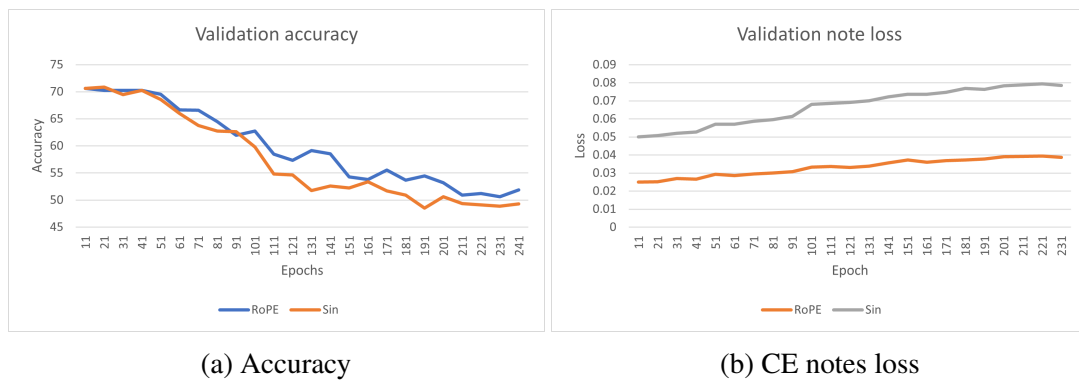


Figure 4.12.: Base model with padding and special character separation - validation graphs. CE - Cross Entropy

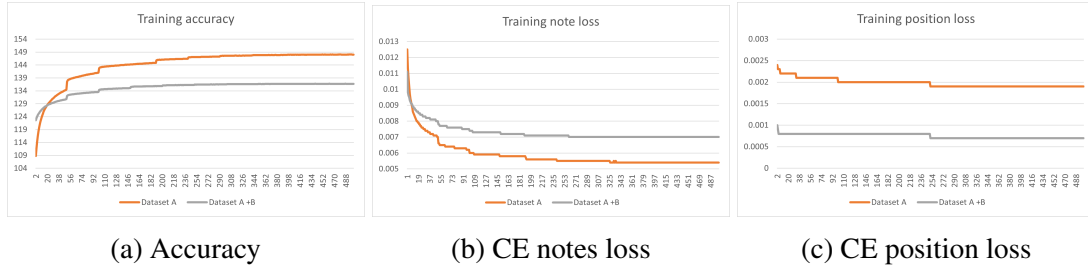


Figure 4.13.: Base model with padding, 0-1 word position information and special character separation - training graphs. CE - Cross Entropy

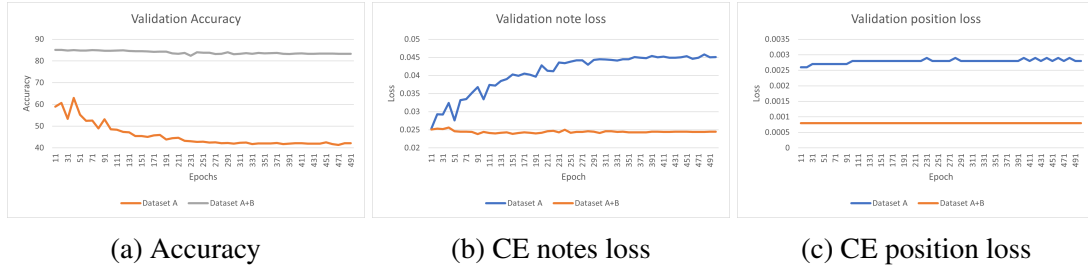


Figure 4.14.: Base model with padding, 0-1 word position information and special character separation - validation graphs. CE - Cross Entropy

For this model, we wanted to see if using a larger dataset would help for the convergence of the model. So, we conducted the comparative study with dry dataset A and another with dry dataset A+B. Dataset A had 12,753 training samples and 574 validation samples, whereas the combined dataset had 70,706 training samples and 6,446 validation samples.

To do a fair analysis, the model was warmed up, RoPE encoding was provided to the BERT embeddings and notes. The word positions were given sinusoidal encodings. Figure 4.13 contains the graphical representation of the training losses (the positional and the melody) and the accuracy. Similarly, Figure 4.14 represents the validation part. On comparing and analysing the graphs, it can be seen that the training accuracy for the combined dataset is low but the validation is more. This indicated that on training on lesser data there is very high chances of overfitting. The same can also be observed when comparing the different losses.

we conclude from these plots that the model has just the sufficient amount of data in the combined dataset such that it has not overfit or underfit. This can be seen as the validation loss for notes or the position for the combined dataset is neither increasing nor decreasing. A possible extension would be to further include more data and check if it is a data problem or a model problem.

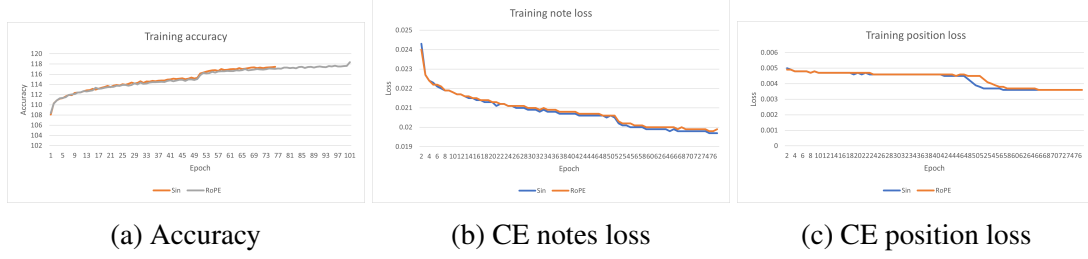


Figure 4.15.: Base model with padding, 0-1 word position information and special character separation for wet data - training graphs. CE - Cross Entropy

4.1.8. Base model with special character separation and 0-1 word positioning with wet dataset

In real-world, vocal music that is available is in the form of wet dataset. This can be obtained by applying the pipeline described in [50] to a music and extracting the vocals from it. This would contain certain artifacts like reverberation or delay, which make it sound better and pleasant to the ear. However, to gain access to the dry vocals which is free from any of these artifacts and is close to the original voice in tune by conversion of the wet vocals, is computationally expensive and time consuming task. Hence, it would make more sense if the model would work on wet data and produce relatable, real-world melody.

Hence, an attempt was made to test the model on wet dataset to see its capability to produce vocal melody. For this purpose, the model was trained on the wet dataset. However, the LyMe pipeline is dependent on the perfect pitch detection, VAD and also the perfect word stamping. These fail to be near perfect as the effects of reverberation, delay and other artifacts tend to hamper all these algorithms and thus, the ground truth is not proper. This results in the decrease in the quality of data. But given that the amount of data is high, it can be assumed that the effects of these artifacts would be less and the model would consider these as noises and discard them.

The note positional encoding was set to sinusoidal encoding while the words' positions' positional encoding was changed between sinusoidal and RoPE encodings. For the other parameters, the BERT embeddings were used and the transformer was not warmed up.

Figure 4.15 and Figure 4.16 shows that with more data there is very low or no overfitting. Also the effect of lower positional loss can also be observed in the accuracy of the training and validation curves. Hence, we can conclude that though the model is capable of handling dry and wet dataset, there is a need for more data in order to gain better results from this model.

4.1.9. Compilation of all models

This subsection compiles all the data from the above 8 models and gives an overview of all the results. It also explains the results. An important thing to note is that the accuracy is calculated as the amount of correct notes predicted given a note. Since the length of each input sequence of notes is 200, then the maximum accuracy that any model can achieve is 199 as the starting

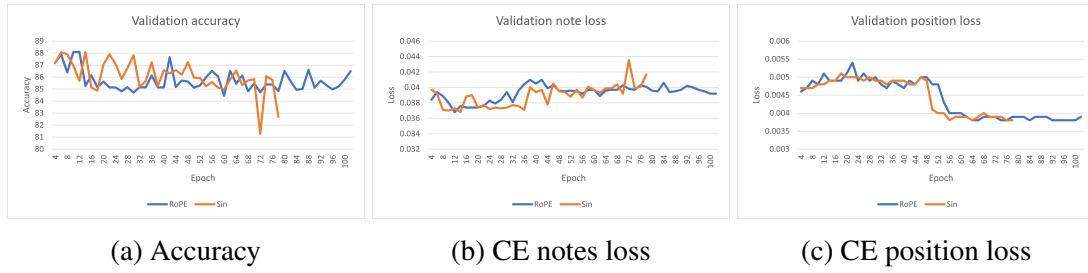


Figure 4.16.: Base model with padding, 0-1 word position information and special character separation for wet data - validation graphs. CE - Cross Entropy

note has to be given to the model.

The Table 4.1 gives the values of the metrics when the best metrics of the training data is taken into consideration. However, for some models, the metrics of the validation data is better than that of the models mentioned in Table 4.1. For Model 2, if the configuration is changed to no warm-up, sinusoidal positional encoding to words and RoPE to the notes then the loss reduces to 0.022 and the accuracy increases to 82.7266, while the training loss and accuracy dips by 0.1%. Similarly if the configuration for model 3 is changed to warm-up with RoPE encoding to the words and sinusoidal encoding to the notes then the validation loss becomes 0.0222 and the accuracy increases to 81.9908, while the numbers for the training remain the same or dip by 0.15%. For model 4, when the dry dataset A is used along with changing all the positional encodings to sinusoidal, the validation loss dips to 0.0923 from 0.0972 and the accuracy increases from 42.45 to 46.74. For model 5, the change is by 1% when the positional encoding of notes and words' positions are interchanged.

The improvements seen in model 1 is due to the fact that there is transfer learning in place. The LAKH dataset does not have the text data but contains accurate information on musical data, whereas the dry dataset A contains accurate text data but not accurate musical information. So the model leans the musical context from the LAKH dataset and tries to transfer the learning by using the dry dataset A. Similarly in model 5, it was trained on a larger corpus of dataset which enabled it learn more and provide better results atleast on the validation data. The 0-1 word positioning also helped overcome the difficulties in the absolute word positioning.

However, these values will change with the change in the quality of dataset. Some of these are a result of the overfitting issue that occurs, while others are due to the fact that the dataset's ground truth is not a perfect ground truth. Other issues are also discussed in Subsection 4.3.

4.2. Application

To make it easily usable for the users without too much hassle and knowhow, an application was created using Gradio. It can also be used for testing out different configurations and checking out different songs and styles based on the different options present in the app. During this app, the user has the flexibility to choose from different sampling techniques like argmax sampling, top-k sampling, random sampling and nucleus sampling. In top-k sampling,

Table 4.1.: Model Parameters and Performance

Model number	Description	Dataset used	Training Note Loss	Training Position Loss	Training Note Accuracy	Validation Note Loss	Validation Position Loss	Validation Note Accuracy	Warmup, Positional Encoding - (Word, Note, Words position)
1	Base model	LAKH + dry dataset A	0.0012	-	158.8619	0.0087	-	63.0679	Yes (BERT, Sin, -)
2	... with padding	dry dataset A + B	0.0068	-	137.7313	0.0256	-	78.704	Yes (BERT, RoPE, -)
3	... with padding and attention loss	dry dataset A + B	0.0068	-	137.9486	0.0239	-	78.6381	Yes (BERT, RoPE, -)
4	... with word positioning	dry dataset A	0.0054	0.0016	148.0061	0.0448	0.0524	42.4453	Yes (RoPE, RoPE, RoPE)
4	... with word positioning	dry dataset A + B	0.0074	0.0008	133.6778	0.0236	0.0176	83.9143	Yes (RoPE, RoPE, RoPE)
5	... with 0-1 word positioning	dry dataset A + B	0.0034	0.0004	138.1013	0.0117	0.0004	84.3111	Yes (Sin, RoPE, Sin)
6	... with special character separation	dry dataset A	0.0058	0.0016	144.044	0.0407	0.0517	49.0488	Yes (Sin, RoPE, Sin)
7	Model 5 + 6	dry dataset A + B	0.007	0.0007	136.7163	0.0252	0.0245	83.2619	Yes (Sin, RoPE, Sin)
7	Model 5 + 6	dry dataset A	0.0054	0.002	148.0365	0.0451	0.0028	42.166	Yes (Sin, RoPE, Sin)
8	Model 5 + 6	wet dataset	0.0188	0.0035	124.2351	0.0435	0.0039	83.637	No (Sin, Sin, Sin)

Figure 4.17.: LyMe created using Gradio

the note is sampled from the top-k most probable tokens from the probability distribution generated by the model. Nucleus sampling, also known as top-p sampling, involves sampling from the smallest subset of the vocabulary whose cumulative probability exceeds a certain threshold, denoted as p .

Also, in addition to the lyrics, the user can set the BPM, can choose the type of positional encoding for the word positioning and notes. A sample of the application can be seen in Figure 4.17. The model internally computes and provides the list of notes. These are then converted to MIDI so that the user can hear the music. This is done at a frequency of 16 kHz and at a velocity of 127. The conversion is done using *pretty midi*. The attention map is also available to the user to see the attention each BERT token is getting. In order for the user to be able to play the notes, the vocal melody is also displayed in terms of notes. Given that the model is capable of producing good music, a confidence score is also generated. This is the value of $avg_energy * 100$ in Algorithm 5. This would help the user generate trust on the model and also understand the performance of the model.

4.3. Issues

There are some major issues that were identified during the course of this project. They are mostly related to data and the processing steps. The amount of dry data for most of the tasks was too small when some complicated approaches like calculation of attention losses and word

positions' losses were included. The amount of wet data seemed to just be sufficient for the task. Also, many vocals had the issue of multiple singers at a given instance of time. Voice diarization would have helped to segregate the voice into different people.

Another aspect might be the variety of data. Since there is incorporation of pitch shifter, some information might be lost due to this. Also, this reduces the significance of some notes and emphasises on some notes more.

Most importantly, the accuracy of identification of the pitches and the VAD algorithms. The accuracy of them has created a lot of issues. Since these account for the backbone of the models' data, having them close to the ground truth is very essential.

5. Summary

The thesis started with the objective to create vocal melody with guidance from lyrics. Given that there are many SOTA algorithms and models which can create music from text, this proposition would lay foundation to another way of creation of vocal music. This thesis showed that it is possible to create vocal music given the lyrics of the song.

Table 4.1 showed that LyMe is indeed possible and can produce good results. It also showed that given well-labeled, large datasets, this approach can also produce good melody. Even when the data had a lot of artifacts like reverberation and delays, the model 8 could perform almost as good as the other models which were trained on better data. In terms of the note accuracy in the validation set, it produces only one wrong note and all others are within 1% of the other metrics. This showed the importance of large, well-labeled dataset.

This project also analysed various pitch detection algorithms and VAD algorithms which were initially designed and created for audio related data. The efficiency and efficacy of them on music dataset was tested during this project.

Given that the base model of the project is a transformer, which in itself is not a novel technique, other components were added to it, such that it could perform in a more desirable manner. It was seen that different combinations of positional encoding techniques provided different insights to the data.

During the course of the project different ideas on how to interact with the data and make it more understandable like utilisation of word positioning, insertion of special characters and training with wet dataset led to understand the nature of the project and also lay foundation to the future works. It also enabled us to find issues which were related to techniques used, the quality of data, the quantity of data and also the variety of data.

The objective was fulfilled with the introduction of Gradio which enabled users to play around with various parameters in the system and generate various types of vocal music. It also produces other outputs which would enable the user to understand the model and the outputs.

With the advent of technologies and other models like diffusion, the task has more potential in the future. Many aspects of this task can be refined in order to create an end product which would be user friendly and innovative. Some of them are: creation of open sourced vocal datasets (wet and dry), perfection of the pitch detection algorithms to design them for vocal dataset, creation of a VAD algorithm to detect the silences and timestamp them correctly, increasing the efficiency of the timestamping of Whisper model such that the timestamping of the Whisper and that of the VAD coincide. Another future work would be to use vocal diarization to separate different singers such that the dataset is more clean and can be used better.

Bibliography

- [1] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh, “Video generation models as world simulators,” 2024. [Online]. Available: <https://openai.com/research/video-generation-models-as-world-simulators> (p. 1)
- [2] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, “Jukebox: A generative model for music,” *arXiv preprint arXiv:2005.00341*, 2020. (p. 1), (p. 21)
- [3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023. (p. 2)
- [4] J. Su, Z. Jin, and A. Finkelstein, “Hifi-gan-2: Studio-quality speech enhancement via generative adversarial networks conditioned on acoustic features,” in *2021 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. IEEE, 2021, pp. 166–170. (p. 2)
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018. (p. 2), (p. 14)
- [6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023. (p. 2)
- [7] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, “Crepe: A convolutional representation for pitch estimation,” in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2018, pp. 161–165. (p. 4), (p. 5), (p. 32)
- [8] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, “Pesto: Pitch estimation with self-supervised transposition-equivariant objective,” in *International Society for Music Information Retrieval Conference (ISMIR 2023)*, 2023. (p. 4), (p. 5), (p. 6), (p. 32)
- [9] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert, “A lightweight instrument-agnostic model for polyphonic note transcription and multipitch estimation,” in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, Singapore, 2022. (p. 4), (p. 6), (p. 7), (p. 32)

- [10] A. De Cheveigné and H. Kawahara, “Yin, a fundamental frequency estimator for speech and music,” *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002. (p. 4)
- [11] A. Camacho and J. G. Harris, “A sawtooth waveform inspired pitch estimator for speech and music,” *The Journal of the Acoustical Society of America*, vol. 124, no. 3, pp. 1638–1652, 2008. (p. 4)
- [12] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, “Rwc music database: Popular, classical and jazz music databases.” in *Ismir*, vol. 2, 2002, pp. 287–288. (p. 4)
- [13] R. M. Bittner, J. Salamon, M. Tierney, M. Mauch, C. Cannam, and J. P. Bello, “Meditdb: A multitrack dataset for annotation-intensive mir research.” in *ISMIR*, vol. 14, 2014, pp. 155–160. (p. 4)
- [14] M. Mauch, S. Ewert *et al.*, “The audio degradation toolbox and its application to robustness evaluation,” 2013. (p. 5)
- [15] D. Chicco, “Siamese neural networks: An overview,” *Artificial neural networks*, pp. 73–94, 2021. (p. 5)
- [16] B. Gfeller, C. Frank, D. Roblek, M. Sharifi, M. Tagliasacchi, and M. Velimirović, “Spice: Self-supervised pitch estimation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 1118–1128, 2020. (p. 5)
- [17] J. Engel, R. Swavely, L. H. Hantrakul, A. Roberts, and C. Hawthorne, “Self-supervised pitch detection by inverse audio synthesis,” in *ICML 2020 Workshop on Self-supervision in Audio and Speech*, 2020. (p. 5)
- [18] S. Team, “Silero models: pre-trained enterprise-grade stt / tts models and benchmarks,” <https://github.com/snakers4/silero-models>, 2021. (p. 8)
- [19] D. Yu, M. Kolbæk, Z.-H. Tan, and J. Jensen, “Permutation invariant training of deep models for speaker-independent multi-talker speech separation,” in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, pp. 241–245. (p. 8)
- [20] H. Bredin, R. Yin, J. M. Coria, G. Gelly, P. Korshunov, M. Lavechin, D. Fustes, H. Titeux, W. Bouaziz, and M.-P. Gill, “Pyannote. audio: neural building blocks for speaker diarization,” in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7124–7128. (p. 9)
- [21] H. Bredin and A. Laurent, “End-to-end speaker segmentation for overlap-aware resegmentation,” *arXiv preprint arXiv:2104.04045*, 2021. (p. 9)
- [22] G. Gelly and J.-L. Gauvain, “Optimization of rnn-based speech activity detection,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 3, pp. 646–656, 2017. (p. 9)

- [23] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” *Advances in neural information processing systems*, vol. 33, pp. 12 449–12 460, 2020. (p. 10)
- [24] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 28 492–28 518. (p. 10)
- [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017. (p. 10), (p. 12), (p. 16)
- [26] M. Kadlčák, A. Hájek, J. Kieslich, and R. Winiecki, “A whisper transformer for audio captioning trained with synthetic captions and transfer learning,” *arXiv preprint arXiv:2305.09690*, 2023. (p. 10), (p. 34)
- [27] R. Olivier and B. Raj, “There is more than one kind of robustness: Fooling whisper with adversarial examples,” *arXiv preprint arXiv:2210.17316*, 2022. (p. 10)
- [28] L. R. S. Gris, R. Marcacini, A. C. Junior, E. Casanova, A. Soares, and S. M. Aluísio, “Evaluating openai’s whisper asr for punctuation prediction and topic modeling of life histories of the museum of the person,” *arXiv preprint arXiv:2305.14580*, 2023. (p. 10)
- [29] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024. (p. 20)
- [30] A. Van Den Oord, O. Vinyals *et al.*, “Neural discrete representation learning,” *Advances in neural information processing systems*, vol. 30, 2017. (p. 21)
- [31] R. Hennequin, A. Khlif, F. Voituret, and M. Moussallam, “Spleeter: a fast and efficient music source separation tool with pre-trained models,” *Journal of Open Source Software*, vol. 5, no. 50, p. 2154, 2020. (p. 21)
- [32] C. Gupta, E. Yilmaz, and H. Li, “Automatic lyrics alignment and transcription in polyphonic music: Does background music help?” in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 496–500. (p. 21)
- [33] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, “Wavlm: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022. (p. 21)
- [34] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “Hubert: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021. (p. 22)

- [35] Q. Huang, A. Jansen, J. Lee, R. Ganti, J. Y. Li, and D. P. Ellis, “Mulan: A joint embedding of music audio and natural language,” *arXiv preprint arXiv:2208.12415*, 2022. (p. 22), (p. 23)
- [36] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, D. Roblek, O. Teboul, D. Grangier, M. Tagliasacchi *et al.*, “Audiolm: a language modeling approach to audio generation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023. (p. 22)
- [37] Y.-A. Chung, Y. Zhang, W. Han, C.-C. Chiu, J. Qin, R. Pang, and Y. Wu, “W2v-bert: Combining contrastive learning and masked language modeling for self-supervised speech pre-training,” in *2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2021, pp. 244–250. (p. 23)
- [38] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, “Soundstream: An end-to-end neural audio codec,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021. (p. 23)
- [39] F. Kreuk, G. Synnaeve, A. Polyak, U. Singer, A. Défossez, J. Copet, D. Parikh, Y. Taigman, and Y. Adi, “Audiogen: Textually guided audio generation,” *arXiv preprint arXiv:2209.15352*, 2022. (p. 24)
- [40] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020. (p. 24)
- [41] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, “Fr\’echet audio distance: A metric for evaluating music enhancement algorithms,” *arXiv preprint arXiv:1812.08466*, 2018. (p. 25)
- [42] S. Forsgren and H. Martiros, “Riffusion - Stable diffusion for real-time music generation,” 2022. [Online]. Available: <https://riffusion.com/about> (p. 25)
- [43] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, “Simple and controllable music generation,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. (p. 25)
- [44] F. Schneider, O. Kamal, Z. Jin, and B. Schölkopf, “Mo[^]usai: Text-to-music generation with long-context latent diffusion,” *arXiv preprint arXiv:2301.11757*, 2023. (p. 26)
- [45] J. Melechovsky, Z. Guo, D. Ghosal, N. Majumder, D. Herremans, and S. Poria, “Mustango: Toward controllable text-to-music generation,” *arXiv preprint arXiv:2311.08355*, 2023. (p. 26)
- [46] A. S. Hussain, S. Liu, C. Sun, and Y. Shan, “M²ugen: Multi-modal music understanding and generation with the power of large language models,” *arXiv preprint arXiv:2311.11255*, 2023. (p. 28)

- [47] “Understanding rope,” <https://medium.com/@ngiengkianyew/understanding-rotary-positional-encoding-40635a4d078e>, accessed: 2024-01-13. (p. 36)
- [48] C. Raffel, *Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching*. Columbia University, 2016. (p. 36)
- [49] M.-T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” *arXiv preprint arXiv:1508.04025*, 2015. (p. 42)
- [50] W.-T. Lu, J.-C. Wang, Q. Kong, and Y.-N. Hung, “Music source separation with band-split rope transformer,” in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 481–485. (p. 43), (p. 60)

A. Appendix

A.1. Mathematical explanation of RoPE

Let $\mathbb{S}_N = \{w_i\}_{i=1}^N$ be a sequence of N input tokens with w_i being the i^{th} element. The corresponding word embedding of $\mathbb{S}_N$ is denoted as $\mathbb{E}_N = \{\mathbf{x}_i\}_{i=1}^N$, where $\mathbf{x}_i \in \mathbb{R}^d$ is the d -dimensional word embedding vector of token w_i without position information. The self-attention first incorporates position information to the word embeddings and transforms them into queries, keys, and value representations:

$$\begin{aligned}\mathbf{q}_m &= f_q(x_m, m) \\ \mathbf{k}_n &= f_k(x_n, n) \\ \mathbf{v}_n &= f_v(x_n, n),\end{aligned}\tag{A.1}$$

where $\mathbf{q}_m$, $\mathbf{k}_n$, and $\mathbf{v}_n$ denote the m th and n th positions through f_q , f_k , and f_v , respectively. The query and key values are then used to compute the attention weights, while the output is computed as the weighted sum over the value representation:

$$\begin{aligned}a_{m,n} &= \frac{\exp\left(\frac{\mathbf{q}_m^\top \mathbf{k}_n}{\sqrt{d}}\right)}{\sum_{j=1}^N \exp\left(\frac{\mathbf{q}_m^\top \mathbf{k}_j}{\sqrt{d}}\right)} \\ \mathbf{o}_m &= \sum_{n=1}^N a_{m,n} \mathbf{v}_n\end{aligned}\tag{A.2}$$

The $a_{m,n}$ represents the attention weights and $\mathbf{o}_m$ represents the output. The existing approaches of transformer-based position encoding mainly focus on choosing a suitable function to form Equation A.1. The algorithm should focus on the the word embedding $\mathbf{x}_m$, $\mathbf{x}_n$ and the relative positioning $m - n$. Hence, it is required to solve the equation :

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = g(\mathbf{x}_m, \mathbf{x}_n, m - n)\tag{A.3}$$

Thus, it becomes imperative to solve and find out the function g in Equation A.3 which maintains the relationship.

The algorithm that was proposed and implemented uses sin and cosine as the trigonometric functions to encode positional information in a different way. To generalize the RoPE in 2D

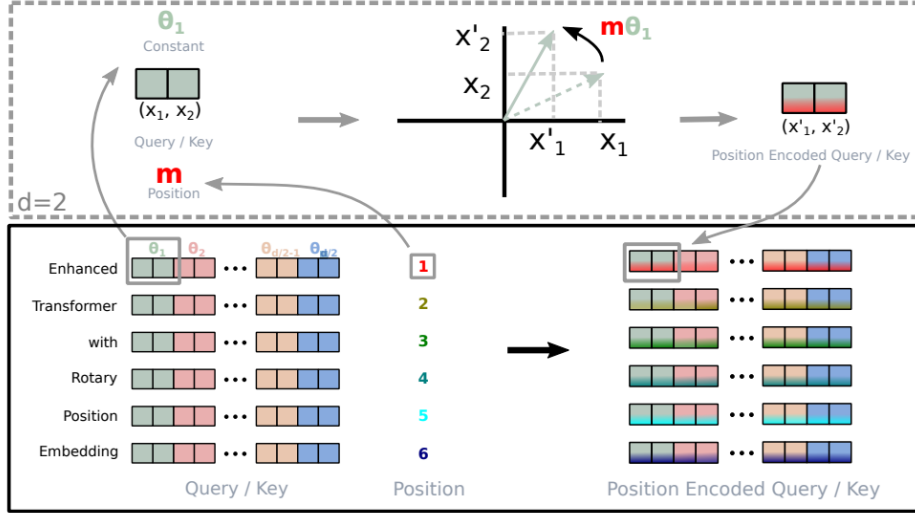


Figure A.1.: Pictographic representation of RoPE [roformer paper]

for any $\mathbf{x}_i \in \mathbb{R}^d$ where d is even, there is a need to turn $f_{q,k}$ into

$$f_{q,k}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{q,k} \mathbf{x}_m \quad (\text{A.4})$$

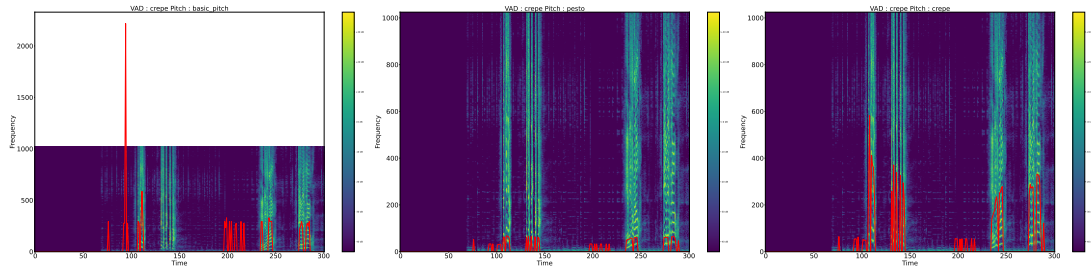
where

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

The matrix $\mathbf{R}_{\Theta, m}^d$ is the rotary matrix with predefined parameters $\Theta = \theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]$. One graphical representation for it is shown in Figure A.1. The self attention term $\mathbf{q}_m^\top \mathbf{k}_n$ transforms to incorporate the matrix $\mathbf{R}_{\Theta, m}^d$ and becomes :

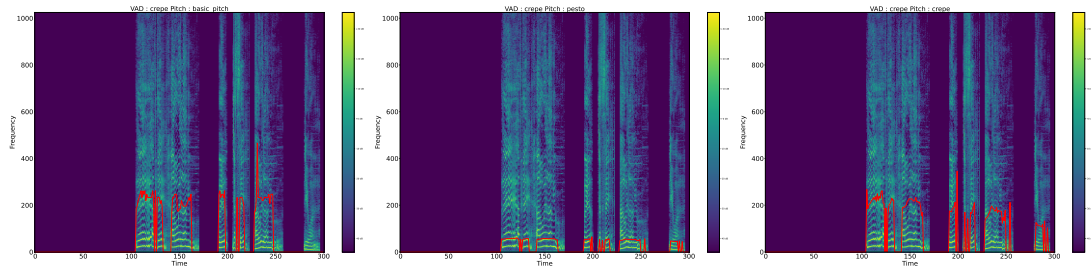
$$\mathbf{q}_m^\top \mathbf{k}_n = (\mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}^\top \mathbf{W}_q \mathbf{R}_{\Theta, n-m}^d \mathbf{W}_k \mathbf{x}_n \quad (\text{A.5})$$

where $\mathbf{R}_{\Theta, n-m}^d = (\mathbf{R}_{\Theta, m}^d)^\top \text{Op} \mathbf{R}_{\Theta, n}^d$. Since $\mathbf{R}_{\Theta}^d$ is an orthogonal matrix, there is stability in the positional encoding. Thus, it was seen that RoPE naturally provides positional information when provided to self-attention mechanism in a transformer.



(a) Pitch map of a song using Basic pitch algorithm (b) Pitch map of a song using PESTO algorithm (c) Pitch map of a song using CREPE algorithm

Figure A.2.: Comparison of pitch detection algorithms for a partial song



(a) Pitch map of a song using Basic pitch algorithm (b) Pitch map of a song using PESTO algorithm (c) Pitch map of a song using CREPE algorithm

Figure A.3.: Comparison of pitch detection algorithms for a partial song

A.2. Pitch detection algorithmic comparison

Here, a few plots could be seen for comparison between different pitch detection algorithms. They are included in Figure A.2 and Figure A.3.

A.3. Model Summary

Table A.1 show the model summary of the basic model.

A.4. Note frequencies

Table A.2 provides the conversion from frequency (Hz) to musical notes.

Modules	Parameters
fc_in.weight	196608
fc_in.bias	256
embedding.weight	28160
transformer_decoder.layers.0.self_attn.in_proj_weight	196608
transformer_decoder.layers.0.self_attn.in_proj_bias	768
transformer_decoder.layers.0.self_attn.out_proj.weight	65536
transformer_decoder.layers.0.self_attn.out_proj.bias	256
transformer_decoder.layers.0.multihead_attn.in_proj_weight	196608
transformer_decoder.layers.0.multihead_attn.in_proj_bias	768
transformer_decoder.layers.0.multihead_attn.out_proj.weight	65536
transformer_decoder.layers.0.multihead_attn.out_proj.bias	256
transformer_decoder.layers.0.linear1.weight	32768
transformer_decoder.layers.0.linear1.bias	128
transformer_decoder.layers.0.linear2.weight	32768
transformer_decoder.layers.0.linear2.bias	256
transformer_decoder.layers.0.norm1.weight	256
transformer_decoder.layers.0.norm1.bias	256
transformer_decoder.layers.0.norm2.weight	256
transformer_decoder.layers.0.norm2.bias	256
transformer_decoder.layers.0.norm3.weight	256
transformer_decoder.layers.0.norm3.bias	256
...	...
fc_out.weight	28160
fc_out.bias	110

Table A.1.: Model Summary

Table A.2.: Note Frequencies

Note	Frequency (Hz)
Silence	0
C0	16.35
C0#	17.32
D0	18.35
D0#	19.45
E0	20.60
F0	21.83
F0#	23.12
G0	24.50
G0#	25.96
A0	27.50
A0#	29.14
B0	30.87
C1	32.70
C1#	34.65
D1	36.71
D1#	38.89
E1	41.20
F1	43.65
F1#	46.25
G1	49.00
G1#	51.91
A1	55.00
A1#	58.27
B1	61.74

Continued on next page

Table A.2 – continued from previous page

Note	Frequency (Hz)
C2	65.41
C2#	69.30
D2	73.42
D2#	77.78
E2	82.41
F2	87.31
F2#	92.50
G2	98.00
G2#	103.83
A2	110.00
A2#	116.54
B2	123.47
C3	130.81
C3#	138.59
D3	146.83
D3#	155.56
E3	164.81
F3	174.61
F3#	185.00
G3	196.00
G3#	207.65
A3	220.00
A3#	233.08
B3	246.94
C4	261.63
C4#	277.18

Continued on next page

Table A.2 – continued from previous page

Note	Frequency (Hz)
D4	293.66
D4#	311.13
E4	329.63
F4	349.23
F4#	369.99
G4	392.00
G4#	415.30
A4	440.00
A4#	466.16
B4	493.88
C5	523.25
C5#	554.37
D5	587.33
D5#	622.25
E5	659.25
F5	698.46
F5#	739.99
G5	783.99
G5#	830.61
A5	880.00
A5#	932.33
B5	987.77
C6	1046.50
C6#	1108.73
D6	1174.66
D6#	1244.51

Continued on next page

Table A.2 – continued from previous page

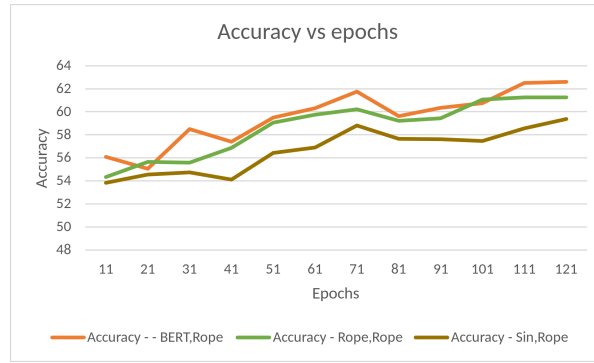
Note	Frequency (Hz)
E6	1318.51
F6	1396.91
F6#	1479.98
G6	1567.98
G6#	1661.22
A6	1760.00
A6#	1864.66
B6	1975.53
C7	2093.00
C7#	2217.46
D7	2349.32
D7#	2489.02
E7	2637.02
F7	2793.83
F7#	2959.96
G7	3135.96
G7#	3322.44
A7	3520.00
A7#	3729.31
B7	3951.07
C8	4186.01
C8#	4434.92
D8	4698.63
D8#	4978.03
E8	5274.04
F8	5587.65

Continued on next page

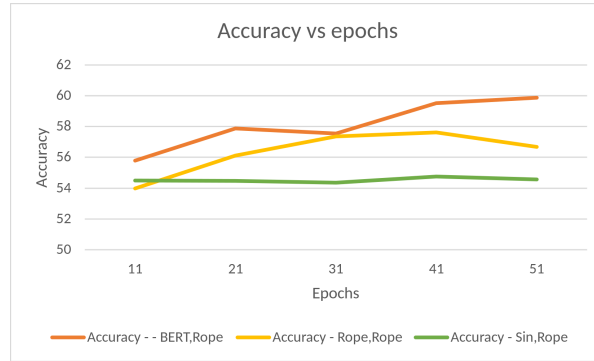
Table A.2 – continued from previous page

Note	Frequency (Hz)
F8#	5919.91
G8	6271.93
G8#	6644.88
A8	7040.00
A8#	7458.62
B8	7902.13

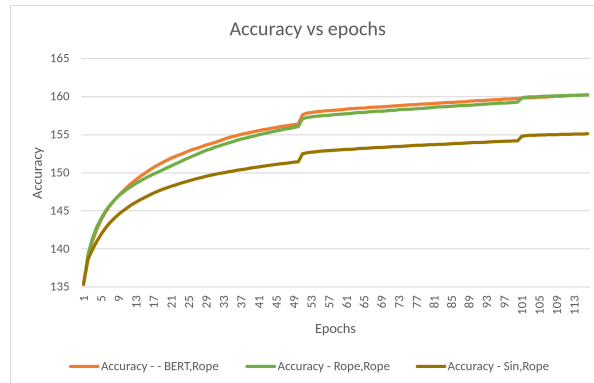
A.5. Base model - Graphs with sinusoidal positioning**A.6. Base model with padding - Graphs with sinusoidal positioning****A.7. Base model with padding and attention mask - Graphs with sinusoidal positioning**



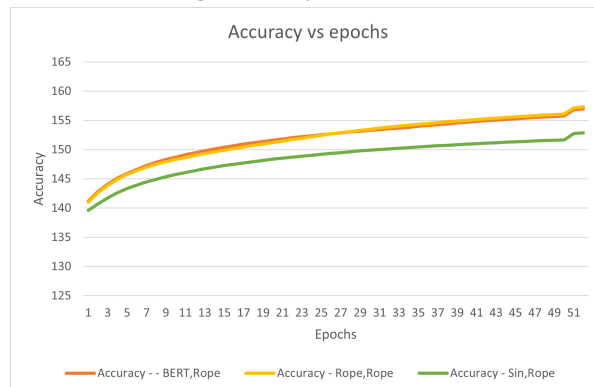
(a) Sampling accuracy with no warm start



(b) Sampling accuracy with warm start

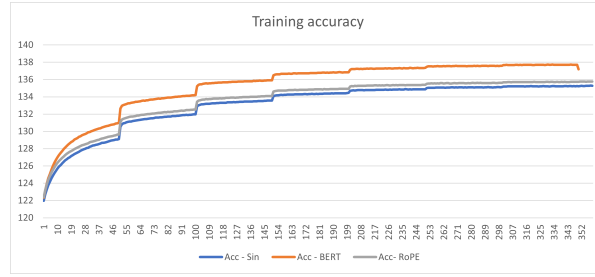


(c) Training accuracy with no warm start

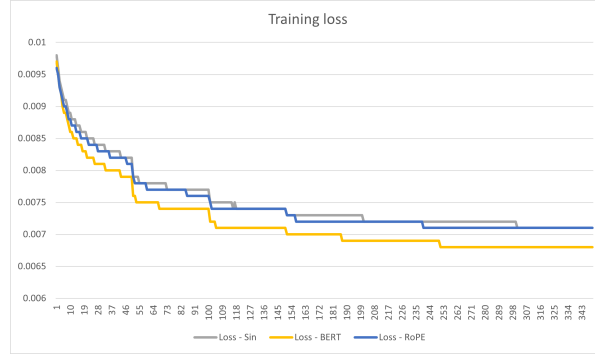


(d) Training accuracy with warm start

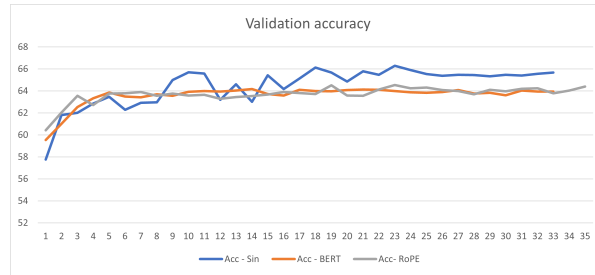
Figure A.4.: Base model training and validation graphs



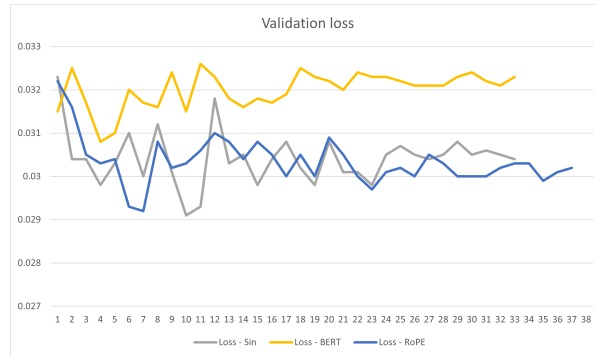
(a) Training accuracy with warm start



(b) Training loss with warm start

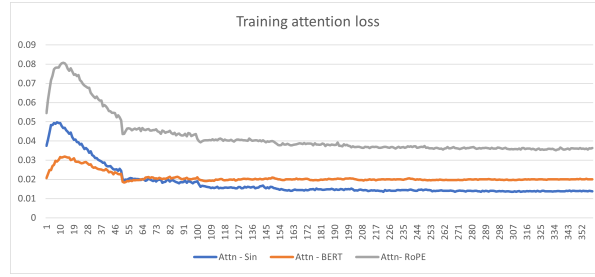


(c) Validation accuracy with warm start

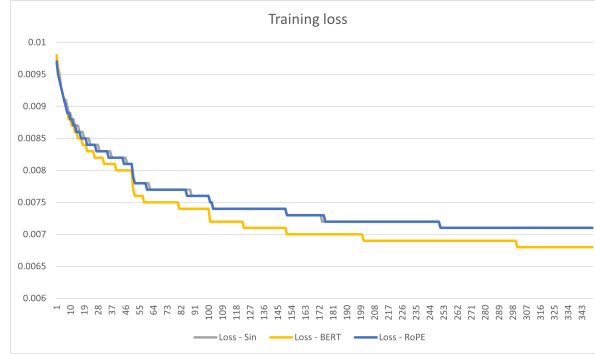


(d) Validation with warm start

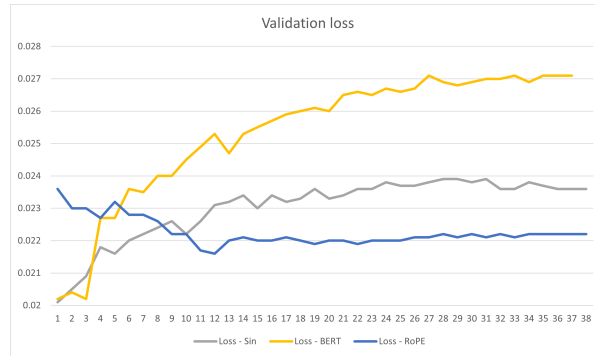
Figure A.5.: Base model with padding - training and validation graphs



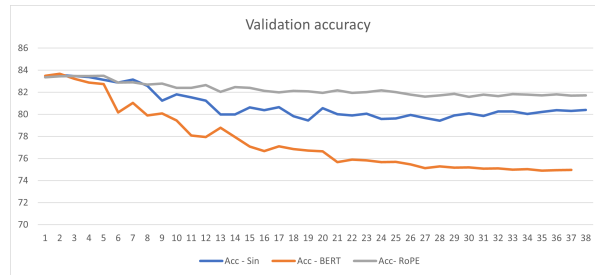
(a) Attention loss during training



(b) Cross entropy loss during training



(c) Total validation loss



(d) Validation accuracy

Figure A.6.: Base model with padding and attention loss - training and validation graphs

Declaration

Herewith, I declare that I have developed and written the enclosed thesis entirely by myself and that I have not used sources or means except those declared.

This thesis has not been submitted to any other authority to achieve an academic grading and has not been published elsewhere.

Stuttgart, 30 April, 2024

Swarnendu Sengupta

Swarnendu Sengupta